US 20250272801A1

(19) **United States**

(12) **Patent Application Publication** (10) **Pub. No.: US 2025/0272801 A1**

Sundaralingam et al. (43) **Pub. Date: Aug. 28, 2025**

(54) **SEEDING MOTION OPTIMIZATION WITH DIFFUSION FOR RAPID MOTION PLANNING**

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA (US)

(72) Inventors: **Balakumar Sundaralingam**, San Jose, CA (US); **Huang Huang**, Berkeley, CA (US); **Arsalan Mousavian**, Seattle, WA (US); **Adithyavairavan Murali**, Seattle, WA (US); **Dieter Fox**, Seattle, WA (US)

(21) Appl. No.: **18/899,363**

(22) Filed: **Sep. 27, 2024**

Related U.S. Application Data

(60) Provisional application No. 63/556,717, filed on Feb. 22, 2024.

Publication Classification

(51) **Int. Cl.**

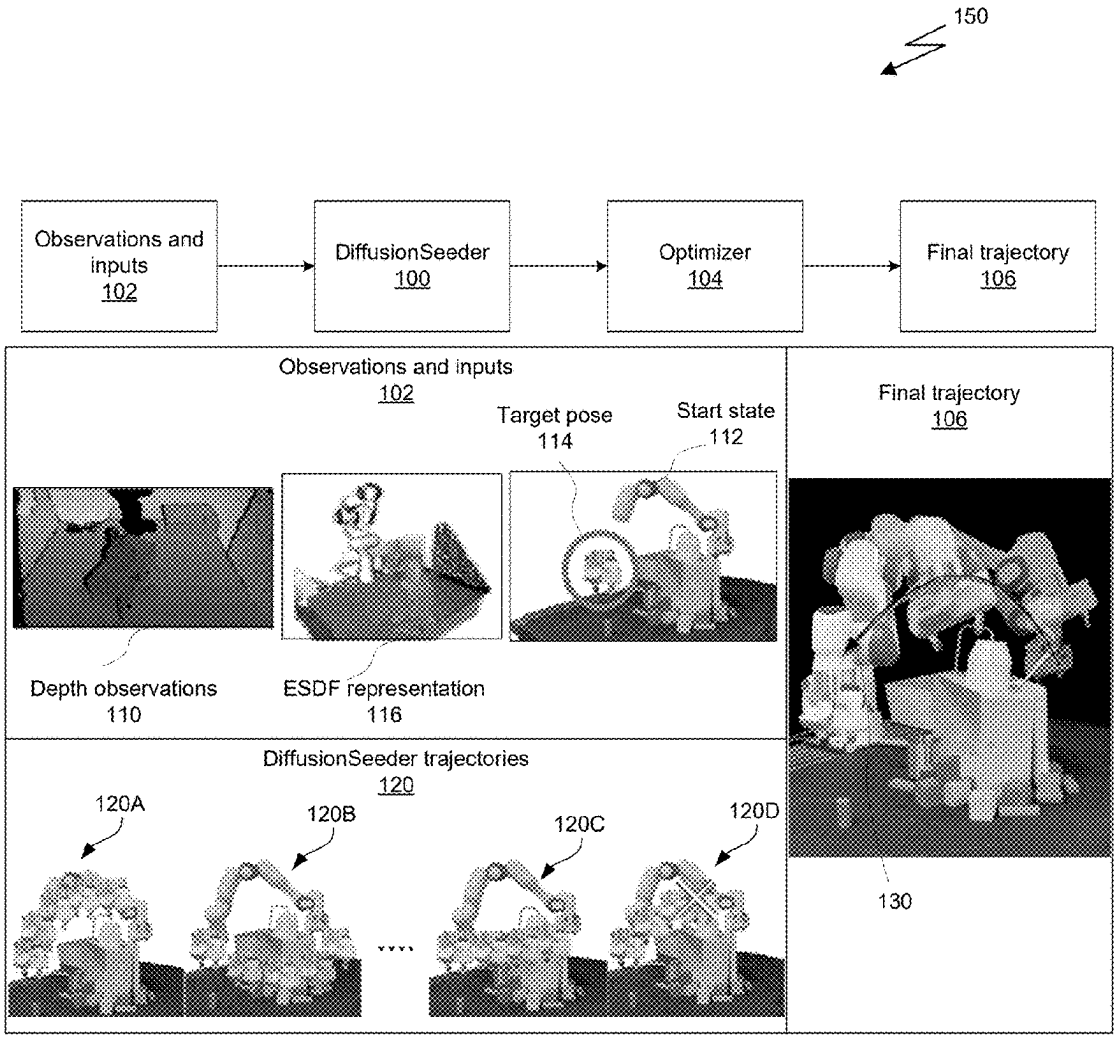
G06T 5/70	(2024.01)
G06T 5/60	(2024.01)
G06T 7/50	(2017.01)

(52) **U.S. Cl.**

CPC ***G06T 5/70*** (2024.01); ***G06T 5/60*** (2024.01); ***G06T 7/50*** (2017.01)

(57) **ABSTRACT**

Embodiments of the present disclosure relate to a neural network architecture for generating high-quality, multi-modal trajectories as seed trajectories for optimization-based motion planners. The neural network architecture utilizes an observation encoder configured to encode environmental observations, and a noise prediction network configured to perform denoising based on the observations. The neural network architecture is configured to generate multiple seed trajectories in parallel by simultaneously running several instances of the noise prediction network. In contrast to conventional techniques, this architecture produces multiple high-quality seed trajectories at once, significantly enhancing the efficiency and speed of motion planning tasks.



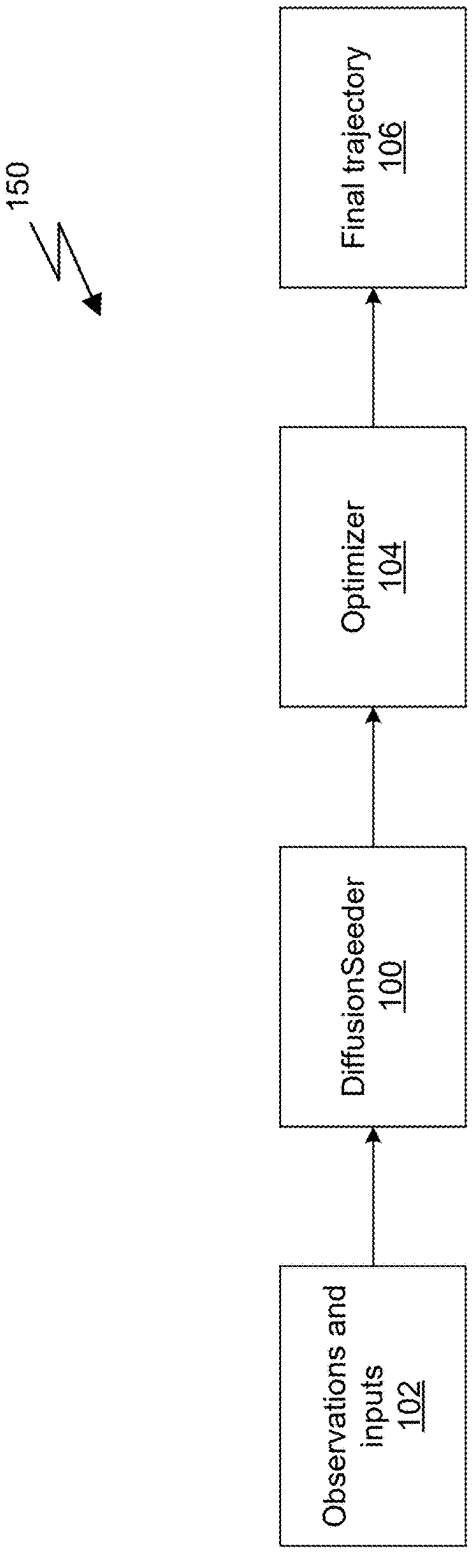

150
Observations and inputs 102
DiffusionSeeder 100
Optimizer 104
Final trajectory 106

Fig. 1A

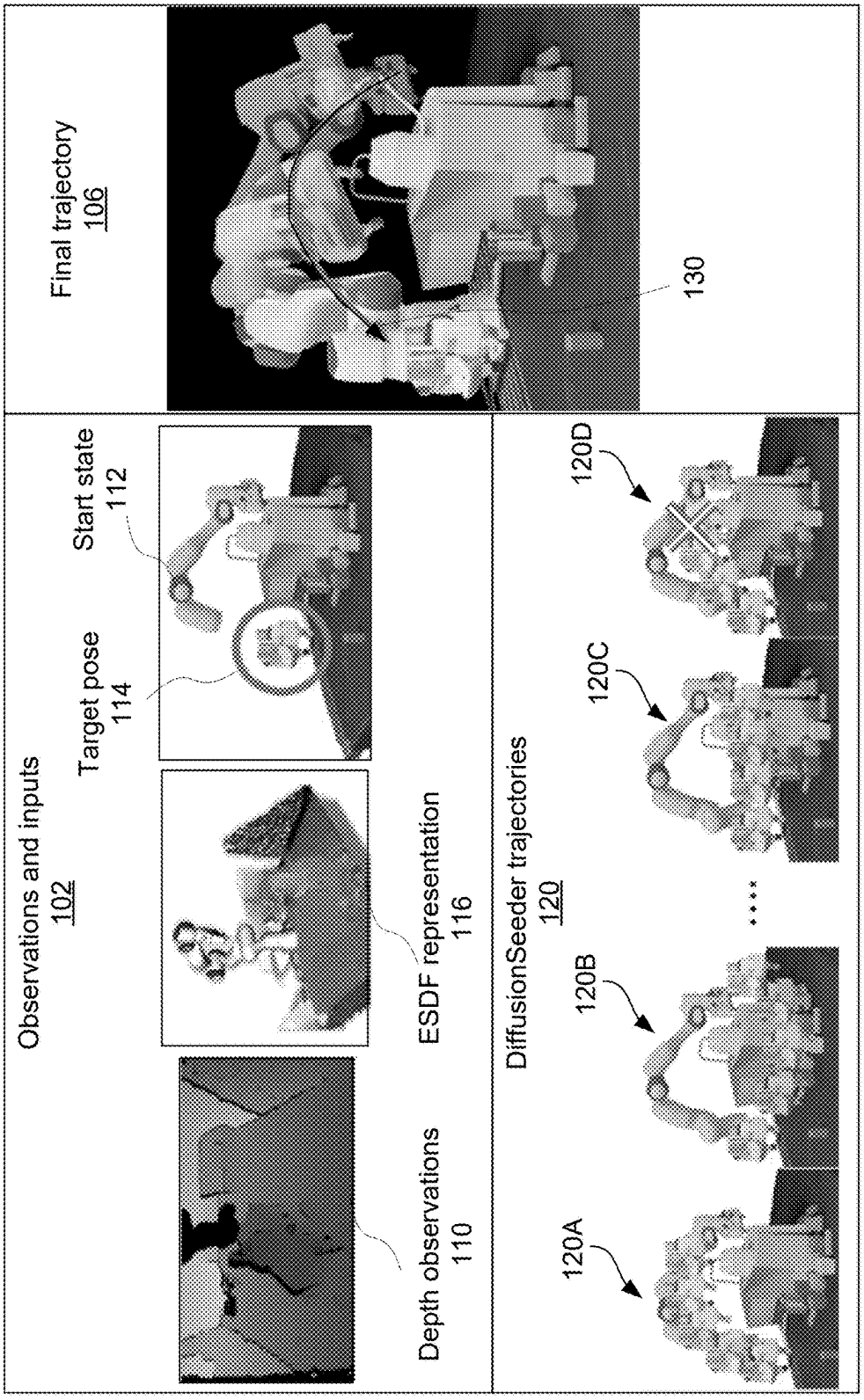

Observations and inputs
102
Depth observations
110
ESDF representation
116
Target pose
114
Start state
112
DiffusionSeeder trajectories
120
120A
120B
120C
120D
Final trajectory
106
130

Fig. 1B

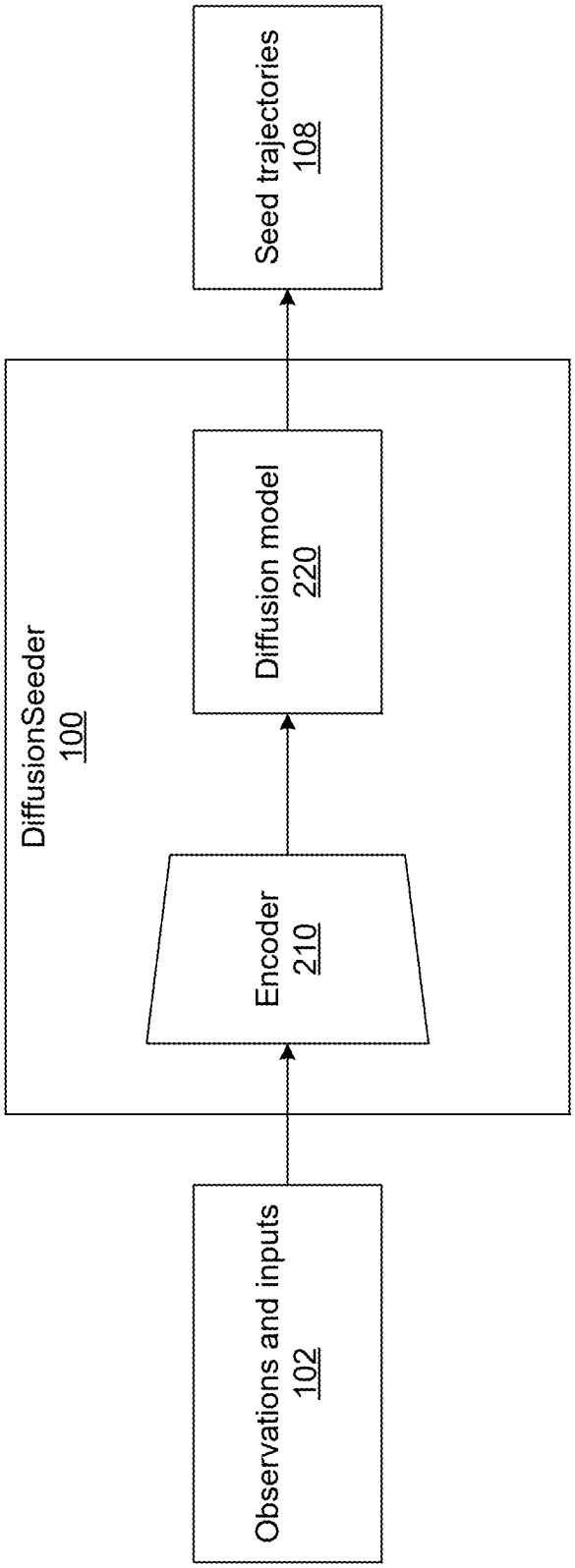

Observations and inputs
102
DiffusionSeeder
100
Encoder
210
Diffusion model
220
Seed trajectories
108

Fig. 2A

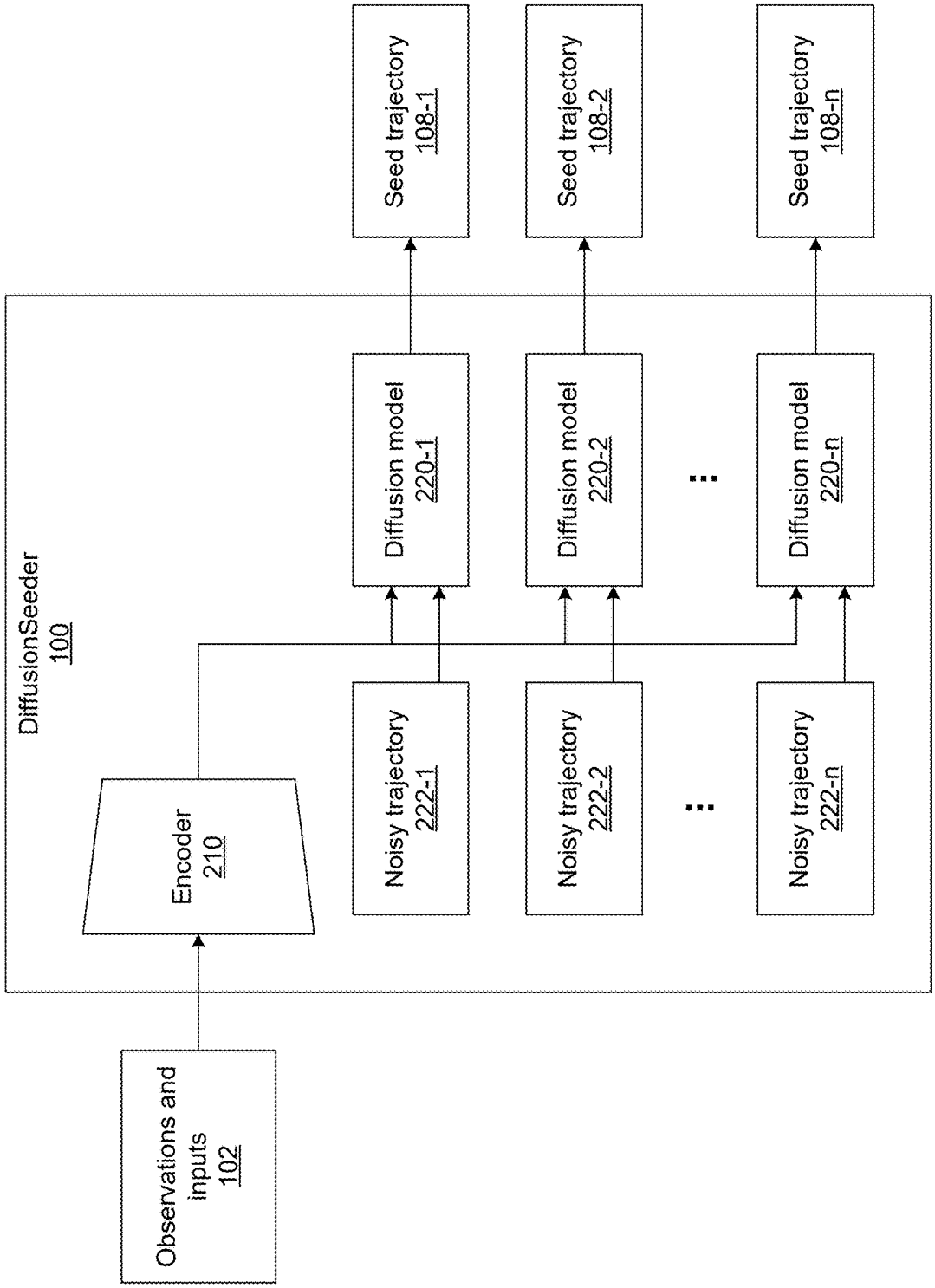

Observations and inputs 102
DiffusionSeeder 100
Encoder 210
Noisy trajectory 222-1
Noisy trajectory 222-2
Noisy trajectory 222-n
Diffusion model 220-1
Diffusion model 220-2
Diffusion model 220-n
Seed trajectory 108-1
Seed trajectory 108-2
Seed trajectory 108-n

Fig. 2B

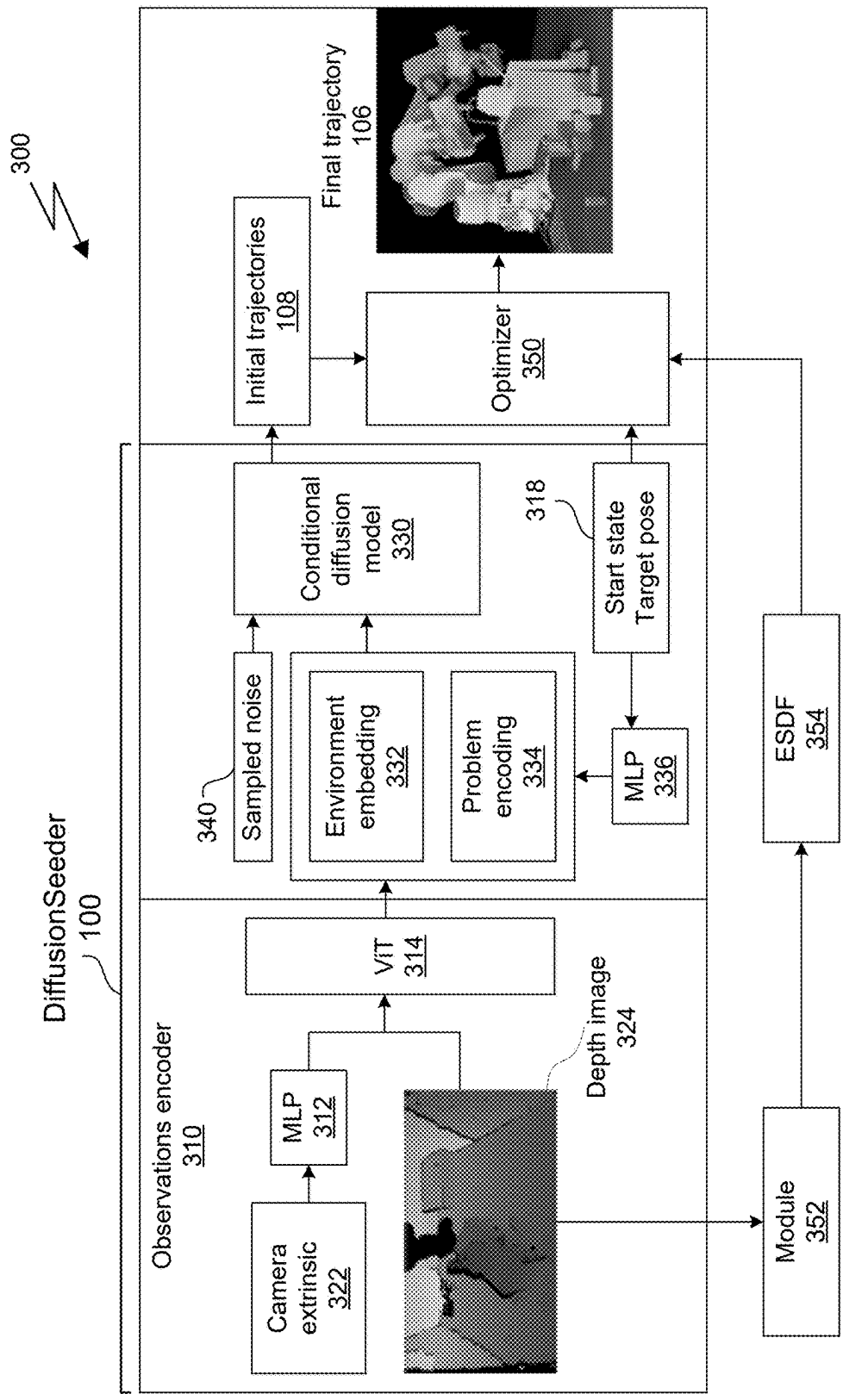

300
DiffusionSeeder
100
Observations encoder
310
Camera extrinsic
322
MLP
312
ViT
314
Depth image
324
Module
352
ESDF
354
340
Sampled noise
Environment embedding
332
Problem encoding
334
MLP
336
318
Start state
Target pose
Conditional diffusion model
330
Initial trajectories
108
Optimizer
350
Final trajectory
106

Fig. 3A

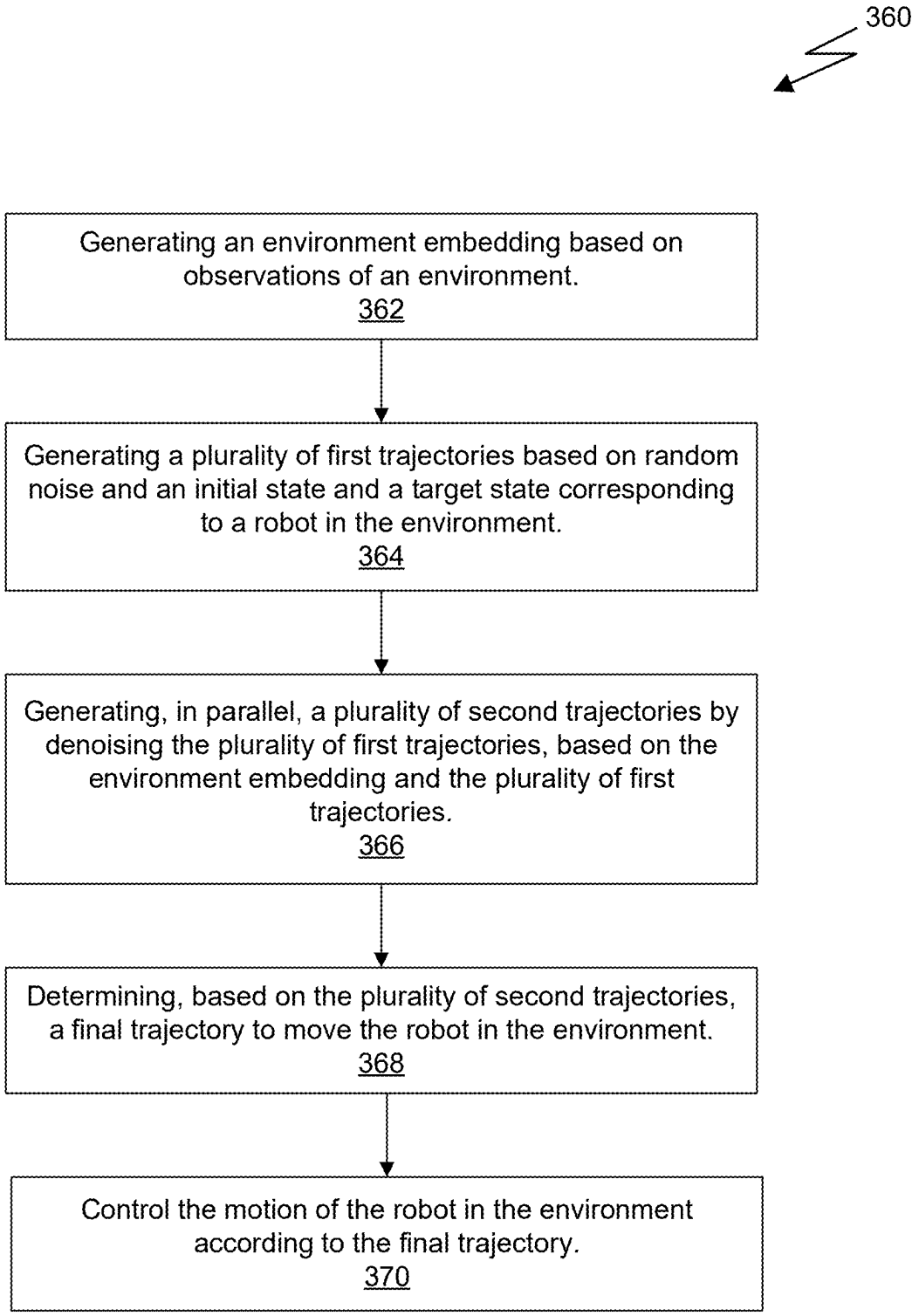
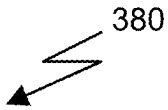

360
Generating an environment embedding based on observations of an environment.
362
Generating a plurality of first trajectories based on random noise and an initial state and a target state corresponding to a robot in the environment.
364
Generating, in parallel, a plurality of second trajectories by denoising the plurality of first trajectories, based on the environment embedding and the plurality of first trajectories.
366
Determining, based on the plurality of second trajectories, a final trajectory to move the robot in the environment.
368
Control the motion of the robot in the environment according to the final trajectory.
370

Fig. 3B



Sampling a ground truth trajectory from a training dataset.
382

Generating a noisy trajectory based on the ground truth trajectory and a random noise.
384

Obtaining, by the encoder in the model, an observation embedding corresponding to the ground truth trajectory.
386

Generating, by the diffusion model in the model, a predicted noise based on the observation embedding and a preset denoising step.
388

Updating learnable weights in the model by minimizing a loss between the random noise and the predicted noise.
390

Fig. 3C

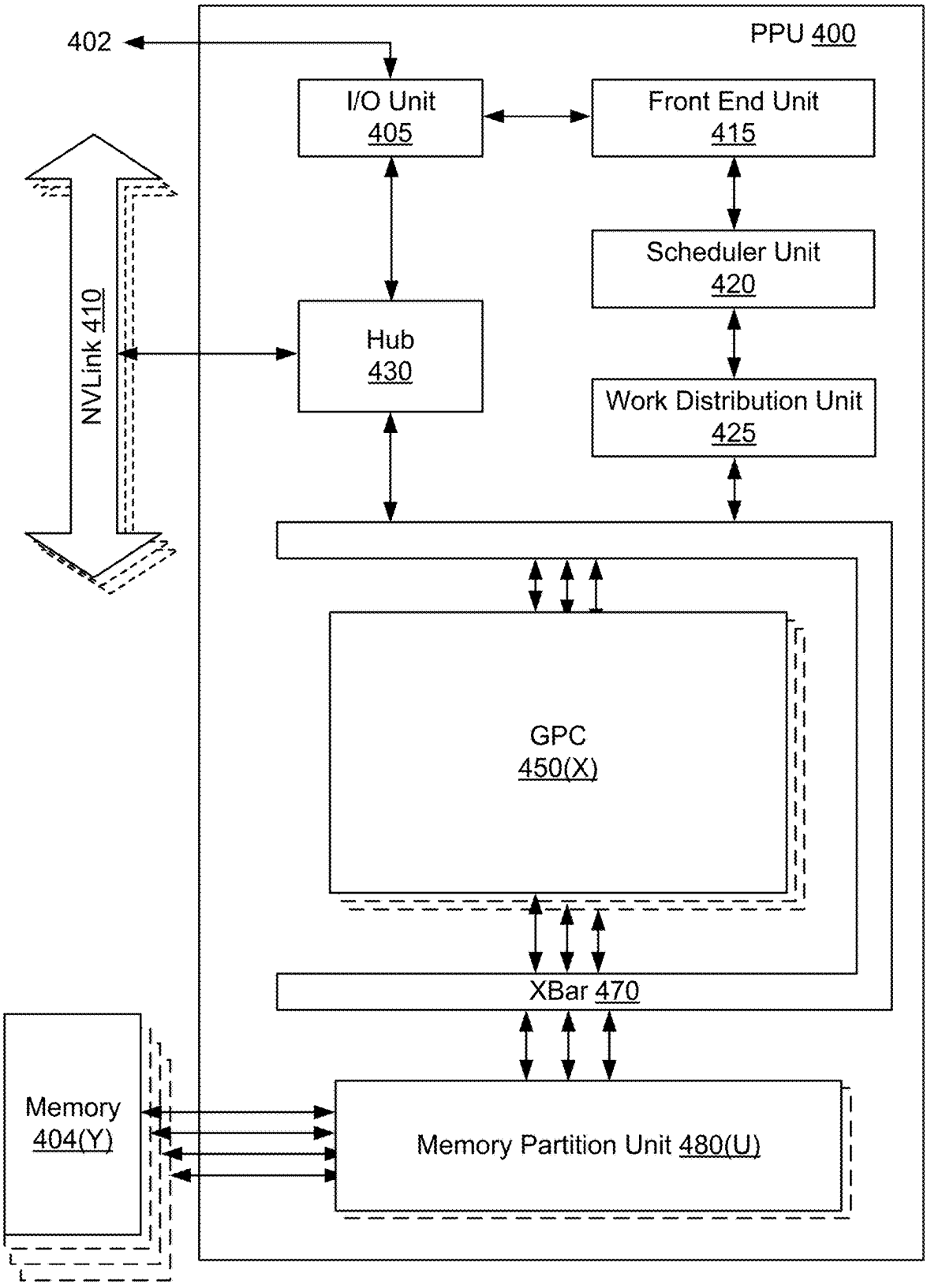

402
PPU 400
I/O Unit 405
Front End Unit 415
Scheduler Unit 420
Work Distribution Unit 425
Hub 430
NVLink 410
GPC 450(X)
XBar 470
Memory 404(Y)
Memory Partition Unit 480(U)

Fig. 4

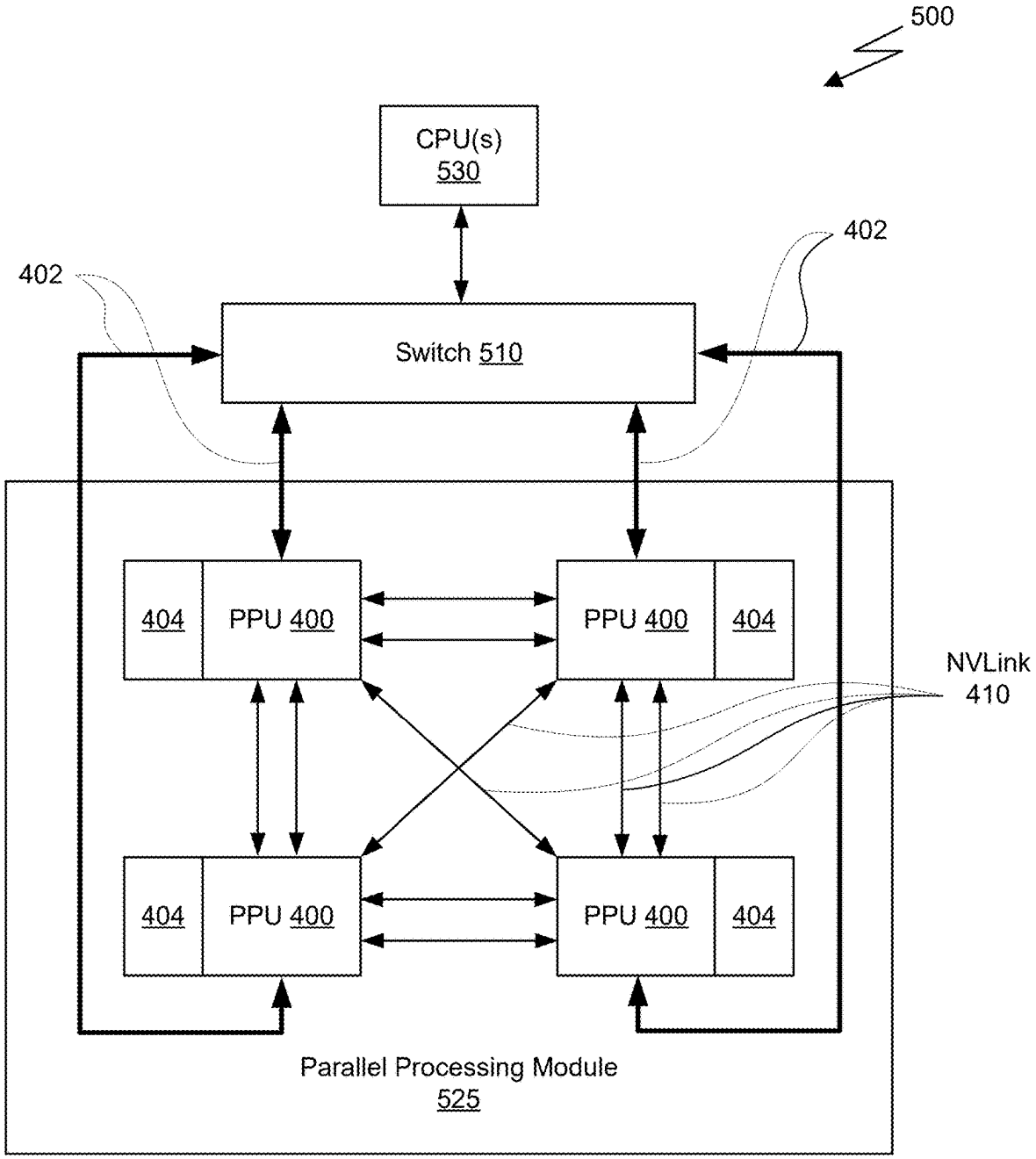

500
CPU(s)
530
402
402
Switch 510
404
PPU 400
PPU 400
404
NVLink
410
404
PPU 400
PPU 400
404
Parallel Processing Module
525

Fig. 5A

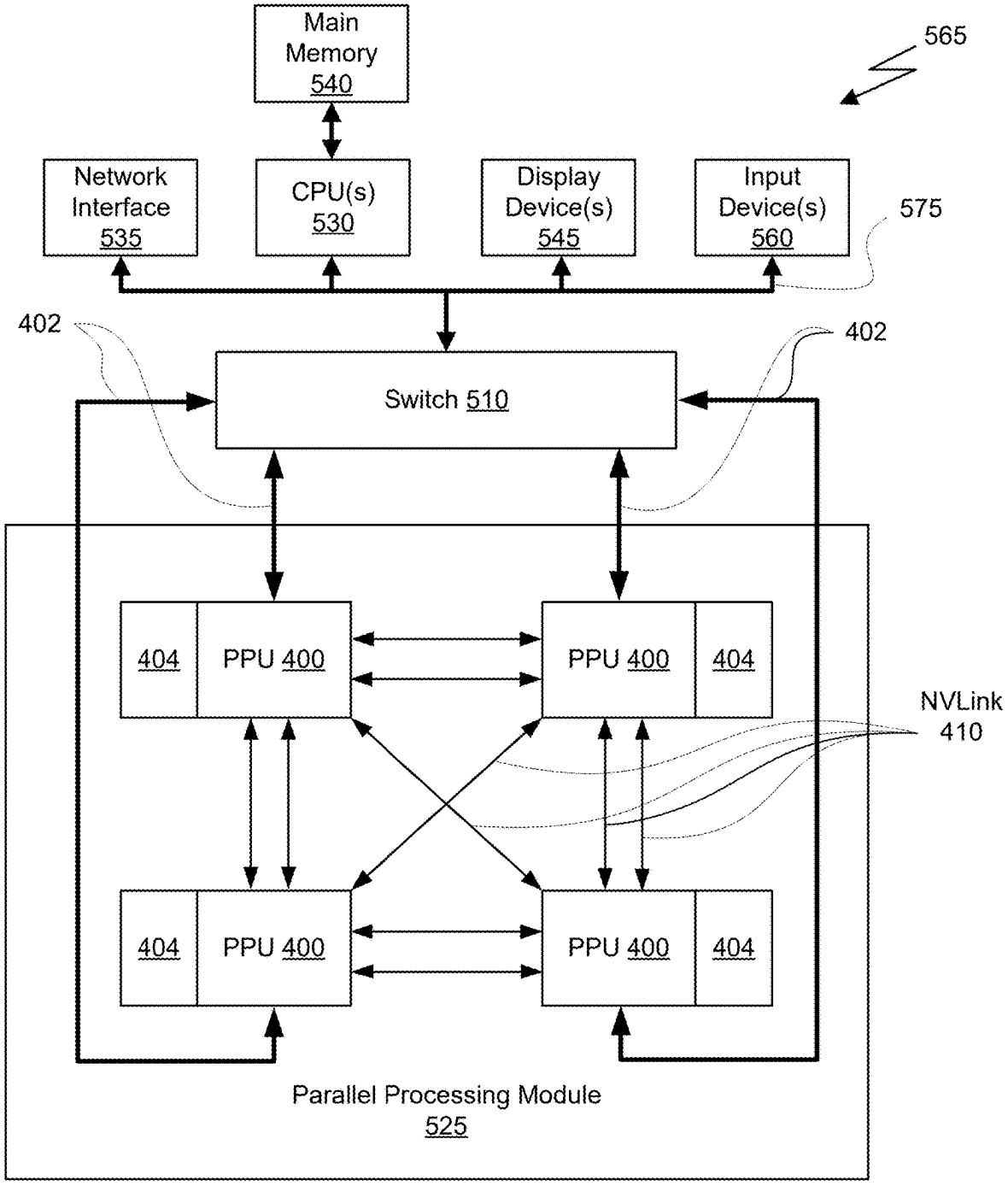

Main Memory 540
565
Network Interface 535
CPU(s) 530
Display Device(s) 545
Input Device(s) 560
575
402
402
Switch 510
404
PPU 400
PPU 400
404
NVLink 410
404
PPU 400
PPU 400
404
Parallel Processing Module 525

Fig. 5B

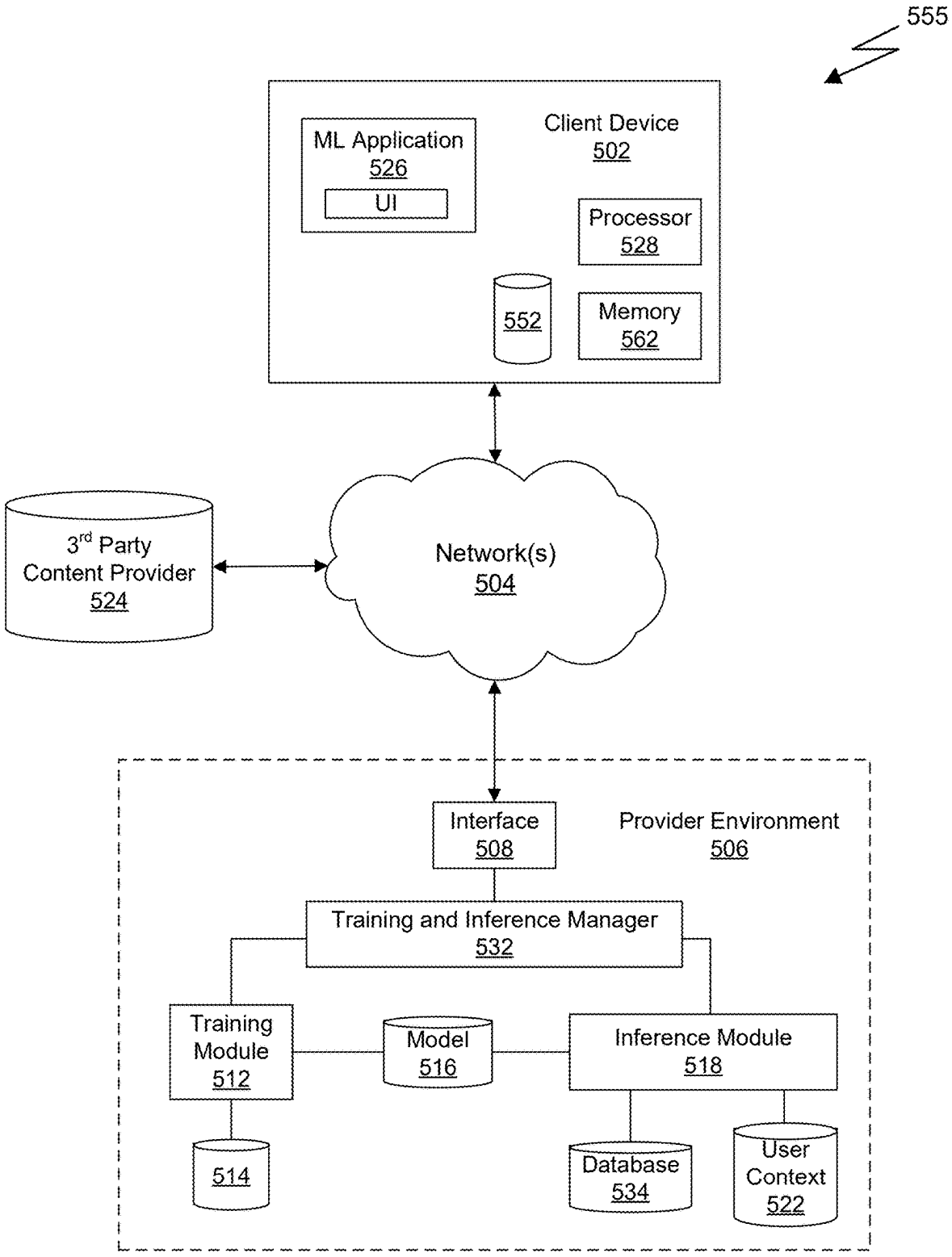

555
Client Device
502
ML Application
526
UI
Processor
528
552
Memory
562
3rd Party
Content Provider
524
Network(s)
504
Provider Environment
506
Interface
508
Training and Inference Manager
532
Training
Module
512
Model
516
Inference Module
518
514
Database
534
User
Context
522

Fig. 5C

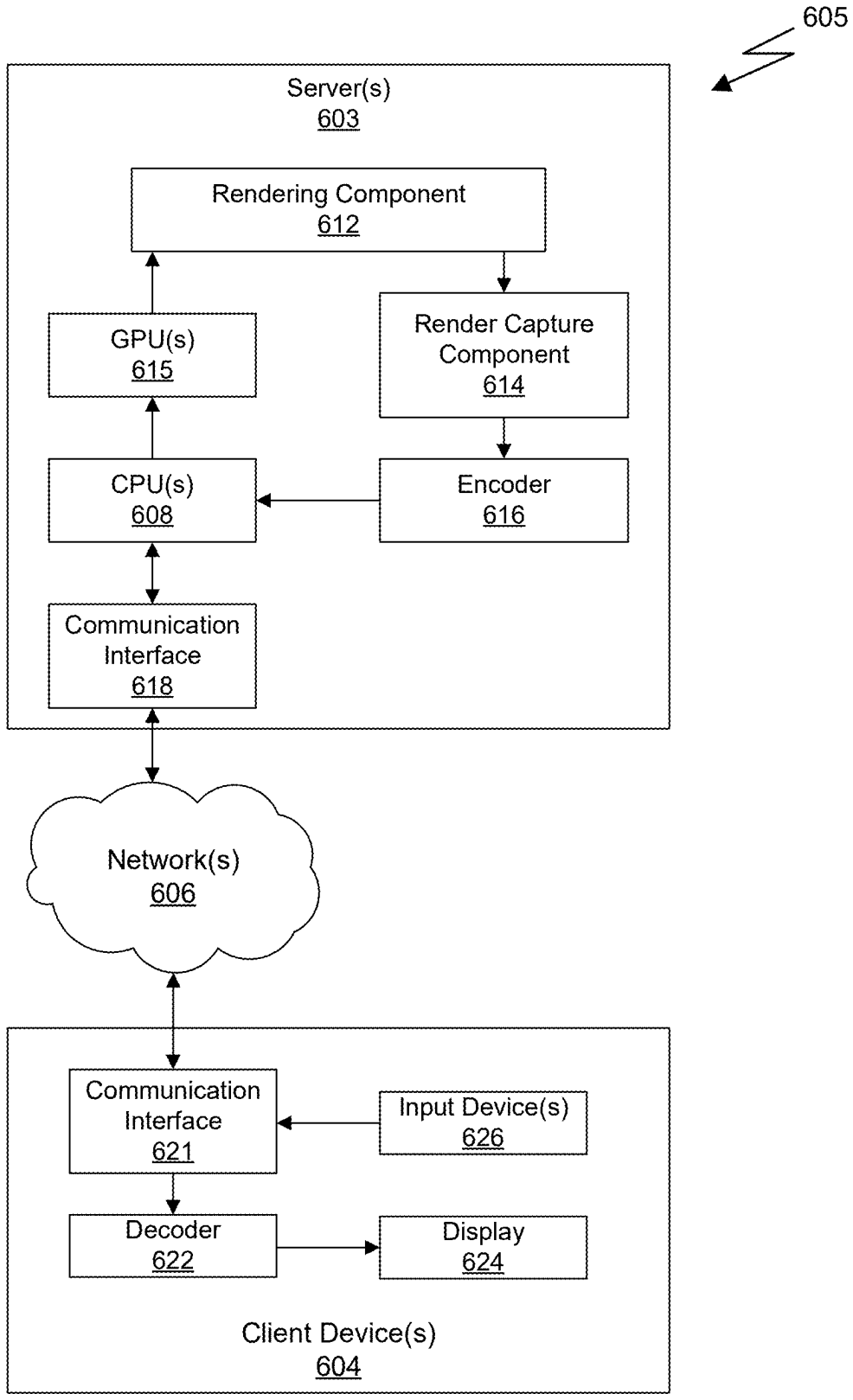

605
Server(s)
603
Rendering Component
612
GPU(s)
615
Render Capture Component
614
CPU(s)
608
Encoder
616
Communication Interface
618
Network(s)
606
Communication Interface
621
Input Device(s)
626
Decoder
622
Display
624
Client Device(s)
604

Fig. 6

SEEDING MOTION OPTIMIZATION WITH DIFFUSION FOR RAPID MOTION PLANNING

CROSS-REFERENCE TO RELATED APPLICATION

[0001] This application claims the benefit of U.S. Provisional Application No. 63/556,717 titled "SEEDING MOTION OPTIMIZATION WITH DIFFUSION FOR RAPID MOTION PLANNING," filed Feb. 22, 2024, the entire contents of which is incorporated herein by reference.

BACKGROUND

[0002] Robot motion generation aims to find a collision-free path in robot configuration space to reach a desired goal pose from a starting configuration. It is a fundamental and challenging problem in robotics due to the high dimensional configuration space, where each configuration corresponds to a combination of robot joint space and a partially-observed representation of the environment. Traditionally, this problem has been solved using sampling-based methods, which sample configurations in the configuration space and check for collisions until a complete path is found. Recently, optimization-based methods, which optimize an initial path under specific objectives and constraints, have also been leveraged to solve this problem. For both of these approaches, planning time increases significantly with the complexity of the configuration space, resulting in a large variance in planning time for different problems. Furthermore, sampling-based methods often require postprocessing to smooth the generated paths, while optimization-based methods, which can generate smooth paths directly, are highly sensitive to the quality of the initial path to be optimized.

SUMMARY

[0003] Embodiments of the present disclosure relate to a neural network architecture for generating high-quality, multi-modal trajectories as seed trajectories for optimization-based motion planners. The neural network architecture utilizes an observation encoder configured to encode environmental observations, and a noise prediction network configured to perform denoising based on the observations. The neural network architecture is configured to generate multiple seed trajectories in parallel by simultaneously running several instances of the noise prediction network. In contrast to conventional techniques, this architecture produces multiple high-quality seed trajectories at once, significantly enhancing the efficiency and speed of motion planning tasks. In at least one embodiment, the observation encoder and the noise prediction network are jointly trained via a joint training procedure.

[0004] According to an aspect of the present disclosure, a system includes one or more processors configured to perform, using one or more neural networks, seed trajectory generation for optimization-based motion planning, and one or more memories to store parameters associated with the one or more neural networks. The one or more neural networks include an observation encoder and a noise prediction network. The observation encoder is configured to: receive a representation of an environment, and generate, by encoding the representation of the environment, an environment embedding. The noise prediction network is configured to: receive the environment embedding, a representation of an initial state, and a representation of a target state, generate a plurality of first trajectories based on the representation of the initial state, the representation of the target state, and random noise, and generate a plurality of seed trajectories by denoising, in parallel, the plurality of first trajectories based on the environment embedding.

[0005] According to an aspect of the present disclosure, a method includes receiving, via an observation encoder network, a representation of an environment; generating, by encoding the representation of the environment via the observation encoder network, an environment embedding; receiving, via a noise prediction network, the environment embedding, a representation of an initial state, and a representation of a target state; generating, via the noise prediction network, a plurality of first trajectories by the noise prediction network and based on the representation of the initial state, the representation of the target state, and random noise; and generating, via the noise prediction network, a plurality of seed trajectories by denoising, by the noise prediction network, the plurality of first trajectories. Denoising the plurality of first trajectories is performed in parallel and based on the environment embedding.

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] The present systems and methods for seeding motion optimization with diffusion for rapid motion planning are described in detail below with reference to the attached drawing figures, wherein:

[0007] FIG. **1**A illustrates a trajectory generation pipeline, in accordance with at least one embodiment.

[0008] FIG. **1**B illustrates embodiments of the trajectory generation pipeline of FIG. **1**A for robot motion planning.

[0009] FIG. **2**A illustrates a neural network architecture for generating a plurality of seed trajectories, in accordance with at least one embodiment.

[0010] FIG. **2**B illustrates a neural network architecture for generating multiple seed trajectories in parallel, in accordance with at least one embodiment.

[0011] FIG. **3**A illustrates a trajectory generation pipeline utilizing neural network architecture for generating a plurality of seed trajectories, in accordance with at least one embodiment.

[0012] FIG. **3**B illustrates a flowchart of a method for motion planning using a neural network architecture for generating a plurality of seed trajectories, in accordance with at least one embodiment.

[0013] FIG. **3**C illustrates a flowchart of a method for training a neural network model for generating a plurality of seed trajectories, in accordance with at least one embodiment.

[0014] FIG. **4** illustrates an example parallel processing unit suitable for use in implementing some embodiments of the present disclosure.

[0015] FIG. **5**A is a conceptual diagram of a processing system implemented using the PPU of FIG. **4**, suitable for use in implementing some embodiments of the present disclosure.

[0016] FIG. **5**B illustrates an exemplary system in which the various architecture and/or functionality of the various previous embodiments may be implemented.

[0017] FIG. **5**C illustrates components of an exemplary system that can be used to train and utilize machine learning, in at least one embodiment.

[0018] FIG. **6** illustrates an exemplary streaming system suitable for use in implementing some embodiments of the present disclosure.

DETAILED DESCRIPTION

[0019] FIG. **1**A illustrates a trajectory generation pipeline **150**, in accordance with at least one embodiment. The pipeline **150** includes various stages that are executed to solve a motion planning problem and generate a final trajectory **106** for motion planning based on observations and inputs **102**. In a first stage, observations and inputs **102** are received and fed into the DiffusionSeeder network **100**. The DiffusionSeeder network **100** is a neural network trained to provide, based on the observations/inputs **102**, a plurality of seed trajectories for an optimization-based motion planner, i.e. optimizer **104**. The plurality of seed trajectories are provided as inputs to the optimizer **104**, which then optimizes the seed trajectories to obtain one or more smooth and collision-free trajectories. A final trajectory **106** is then output based on the results provided by the optimizer **104**.

[0020] FIG. **1**B illustrates embodiments of the trajectory generation pipeline of FIG. **1**A for robot motion planning. The observations and inputs **102** can include various types of data, such as one or more images of an environment, camera properties, states of a robot (e.g., an initial state and a target pose), and other suitable representations. The one or more images of the environment can include color and/or depth images. In at least one embodiment, the one or more images are acquired by various cameras positioned at different locations in the environment, on the robot (e.g., on a suitable link), or a combination thereof. In at least one embodiment, the one or more images are extracted from video frames. In at least one embodiment, the input to the DiffusionSeeder network **100** includes a composite image that is obtained by merging multiple raw images. The camera properties can include suitable parameters to indicate the state information at the time of capture for a corresponding image. For example, the camera properties can include coordinates to indicate the camera's position, a pose matrix to represent the pose information of a dynamic camera during capture, timestamp data, and other relevant information. In at least one embodiment, the camera properties include calibrated intrinsic and extrinsic matrices of the camera. The states of the robot can include relevant information to define the problem for motion planning. For example, motion planning for the robot can be defined by an initial state and a target state, with the motion planning process configured to output intermediate state(s) that bring the robot from a starting configuration (e.g., corresponding to the initial state) to a goal configuration (e.g., corresponding to the target state). The state of the robot can be described by the positions of its joints and/or links, as well as the pose of its end effector. In at least one embodiment, the observations and inputs **102** include other types of representations of the environment, such as a dynamic Euclidean signaled distance field (ESDF) representation **116**.

[0021] Referring to FIG. **1**B, observations and inputs **102** include depth observations **110**, ESDF representation **116**, a robot start configuration **112**, and a target pose **114** of the end state of effector as input to the DiffusionSeeder network **100**. The depth observation **110** may include one or more depth images. The robot start configuration **112** may be represented by a vector, with elements of the vector corresponding to the positions of the robot's joints. The target pose **114** may be represented as a vector and/or matrix, indicating the position and orientation of the end effector.

[0022] In at least one embodiment, the DiffusionSeeder network **100** generates diverse trajectories **120** (e.g., in **120**A-**120**D) based on the observations and inputs **102**. Some trajectories may be in collision. For example, in **120**D, the robot collides with an object (e.g., a teapot) placed on the table.

[0023] The optimizer **104** optimizes the trajectories **120** from the DiffusionSeeder network **100** to determine a final trajectory **106**. The optimizer **104** is, in at least one embodiment, an optimization-based motion planner. In at least one embodiment, the optimizer **104** is configured to use the ESDF representation **116** to move trajectories out of collision, resulting in a smooth, fast, and collision-free final output (e.g., indicated by arrow **130**).

[0024] FIG. **2**A illustrates a neural network architecture for generating a plurality of seed trajectories, in accordance with at least one embodiment. The neural network architecture is the Diffusion Seeder network **100**. As shown in FIG. **2**A, the DiffusionSeeder network **100** includes various functional modules, including an encoder **210** and a diffusion model **220**. The encoder **210** applies one or more suitable networks, such as multi-layer perceptron (MLP), transformer, and/or other types of networks, to encode the observations and inputs **102** of the environment. This process generates an environment embedding and a problem encoding. For example, the encoder **210** generates the environment embedding based on the observations of the environment and the problem encoding based on an initial configuration of the robot and a target pose of an effector attached to the robot.

[0025] The diffusion model **220** processes the environment embedding and the problem encoding to generate one or more seed trajectories **108**. In at least one embodiment, the diffusion model **220** includes a diffusion pipeline that gradually refines an output through a series of denoising steps. The environment embedding is associated with static environment information, while the problem encoding, along with random noise, serves as a noisy trajectory that is iteratively updated and improved across multiple iterations of the denoising steps.

[0026] In at least one embodiment, the DiffusionSeeder network **100** generates an initial trajectory (e.g., a noisy trajectory) based on the problem encoding from the encoder **210** and random noise. Then, the DiffusionSeeder network **100** uses the diffusion model **220** to produce a denoised trajectory, which serves as a seed trajectory **108**.

[0027] In at least one embodiment, the observations and inputs **102** of the environment includes a depth image, properties of a camera, and states of the robot. In at least one embodiment, the depth image has dimensions H×W, where H represents the number of pixels in height and W represents the number of pixels in height width. In at least one embodiment, the properties of the camera include intrinsic and extrinsic matrices. In at least one embodiment, the states of the robot include a starting state of the robot in joint space, represented by q_0, and a final goal pose of the robot end effector, represented as a multi-dimensional vector consisting of a translation and quaternion in Cartesian space. In at least one embodiment, the dimensions of the vector are defined based on the number of joints in the robot. The

motion planning output (e.g., the final trajectory **106**) is represented by τ, which enables the robot to reach the target end-effector pose from the starting state.

[0028] In at least one embodiment, the DiffusionSeeder network **100** is configured to generate diverse, high-quality seed trajectories **108** to speed up a motion optimizer (e.g., the optimizer **104**) to rapidly generate a collision-free trajectory τ in robot joint configuration space that reaches a target end effector pose with position and rotation errors below given thresholds (e.g., δ_t and δ_r, respectively), while minimizing the jerk and motion time of the generated trajectory. In at least one embodiment, the trajectory τ is defined as $\{(t_i,q_i)|i\in[0,T]\}$, where t_i represents a time step, q_i represents a robot joint state at time step t_i, and T represents the length of the trajectory τ.

[0029] In at least one embodiment, the DiffusionSeeder network **100** is configured to generate smooth and collision free seed trajectories **108** with a fixed trajectory length T=32 for a robot with seven joints. The robot joint state q_i is represented by a seven-dimensional vector. Based on this configuration, the DiffusionSeeder network **100** generates the seed trajectories **108** based on noisy trajectories of the same size, each noisy trajectory being represented by a noise tensor with 32 time steps and seven-dimensional state vectors. It will be understood by people skilled in the art that the size and dimensions herein are merely examples, and other suitable numbers may be used depending on the circumstances. In certain embodiments, while the noise tensor is designed with a fixed trajectory length, such as T=32, the joint state represented in the tensor does not necessarily converge (e.g., reach the target state) at the final time step (e.g., the 32^{nd} time step). Instead, the joint state can converge at an earlier time step (e.g., the 10^{th} time step) and stabilize afterward (e.g., the joint states corresponding to the later time steps remain substantially the same).

[0030] In at least one embodiment, the diffusion model **220** utilizes a conditional noise prediction network to facilitate the denoising process. In at least one embodiment, the conditional noise prediction network includes multiple layers executed sequentially. Each layer's input is generated by concatenating an embedding of a recently updated trajectory and the environment embedding from the encoder **210**. The output of the respective layer is a prediction of noise for the current input. The predicted noise is then extracted from the input trajectory to obtain an updated one. These multiple layers enable iterative denoising steps to enhance the quality of the denoised trajectory. As such, executing an instance of the diffusion model **220** yields the output of a high-quality seed trajectory **108** conditioned by the observation.

[0031] FIG. **2**B illustrates a neural network architecture for generating multiple seed trajectories in parallel, in accordance with at least one embodiment. The neural network architecture is the Diffusion Seeder network **100**. As shown in FIG. **2**B, multiple instances (**220-1**, **220-2**, . . . , **220-*n***) of the diffusion model **220** are executed in parallel to perform denoising on noisy trajectories (**222-1**, **222-2**, . . . , **222-*n***) to generate the corresponding seed trajectories (**108-1**, **108-2**, . . . , **108-*n***). During inference, the noisy trajectories (**222-1**, **222-2**, . . . , **222-*n***) are, in at least one embodiment, generated based on the problem encoding and n instances of random noise. During training, the noisy trajectories (**222-1**, **222-2**, . . . , **222-*n***) are, in at least one embodiment, generated based on a ground-truth trajectory and n instances of random noise.

[0032] As discussed above, the observations and inputs **102** may take various forms. The DiffusionSeeder network **100** may operate in combination with various types of optimizers. Since the DiffusionSeeder network **100** is capable of providing multiple seed trajectories at once, when used in combination with an optimizer (e.g., cuRobo) capable of parallel computing, such combination can maximize the advantages of parallel processing. cuRobo is a GPU-accelerated motion optimizer capable of solving a motion planning problem on many seeds in parallel. Given a starting robot configuration and a target end pose, standard cuRobo first computes collision-free inverse kinematics (IK) solutions, and then runs trajectory optimization with linear interpolations from start configuration to these IK solutions. In the case of failure, standard cuRobo falls back to a graph-based motion planner to generate new seeds. As all stages are accelerated through GPU parallelization, cuRobo can generate plans within 64 milliseconds on average. However, on complex scenes where a non-linear trajectory is often needed, cuRobo may experience a severe slowdown in planning speed due to invocation of the graph-based planner, taking up to 0.5 seconds. In at least one embodiment, the multiple seed trajectories provided by the DiffusionSeeder network **100** eliminate the need to compute, via the optimizer, collision-free IK solutions. In at least one embodiment, by integrating the DiffusionSeeder network **100** with cuRobo, a 12× average speed up over standard cuRobo (and a 36× speed up at 98th quantile) was achieved, along with a 6.5% higher success rate on partially occluded views of a scene.

[0033] Various datasets may be generated to train some or all of the modules in the DiffusionSeeder network **100**. In at least one embodiment, a training dataset is generated using cuRobo for a seven Degrees of Freedom (7-DoF) robot on MπNet training scenes. MπNet consists of three million unique scenes in three categories: cubby, tabletop and dresser. A motion planning problem in the MπNet dataset consists of a scene mesh geometry, along with a start and end pose. In at least one embodiment, a training dataset is generated by sampling additional pairs of start and end state poses near the original start and end poses to increase diversity. Since most MπNet training scenes have obstacles near the end pose, problems near the end pose to encourage difficult trajectory generation can also be sampled. In at least one embodiment, a training dataset is generated by obtaining, for each scene, the start pose in Cartesian space from the MπNet start state, and sampling N−1 new motion planning problems (start and end pose pairs) within a bounding box of 0.3 meters along the X-, Y-, and Z-axes around the original pose. Furthermore, in at least one embodiment, N new problems are sampled within a bounding box of 0.3 meters near the end pose. In at least one embodiment, N pairs of poses in free space are sampled, within a cuboid of length 60 centimeters, to increase the coverage over the joint configuration space of the robot. In at least one embodiment, this results in 3N start and end pose pairs for each scene. Inverse kinematics are run for each pose in the problem to filter out pairs where either pose is in collision with the scene. In at least one embodiment, each motion planning problem is passed to cuRobo to generate smooth and collision free trajectories with a fixed trajectory length T=32. Problems for which cuRobo is unable to find a feasible solution are filtered out and discarded. According to at least one embodiment, N=5 and a dataset with 15 million prob-

lems was generated with feasible and smooth solutions of size 32×7 on three million unique scenes.

[0034] In at least one embodiment, environment rendering is performed. To provide partial observations to the planner, depth images are rendered for each scene using a fixed virtual camera located within the scene. In at least one embodiment, each camera pose is represented by an azimuth (a), an elevation (e), and a radius (r). For each scene, orientations are discretized to a fixed grid of $M=N_a \cdot N_e$, where N_a is discretization resolution on azimuth, N_e is discretization resolution on elevation, and M is the number of camera poses on a sphere centered at a scene mesh and pointing inward toward the center of the scene. In at least one embodiment, the azimuth ranges between $[-\pi/4, \pi/4]$, the elevation ranges between $[\pi/5, 2\pi/5]$, and the radius (r) ranges between [0.7×R, 1.1×R], where R is the radius of scene mesh. In at least one embodiment, uniform random noise $\epsilon_a \in [-\pi/8, \pi/8]$ and $\epsilon_e \in [-\pi/15, \pi/15]$ is applied to each discretized a and e, respectively, to obtain diverse camera poses across different scenes. In at least one embodiment, depth images from each camera pose are rendered and camera poses where the problem end pose is not visible in the rendered depth images are filtered out. In at least one embodiment, the goal is visible if the projection of the goal in the camera view has larger depth than the goal itself—in other words, none of the observed pixels can occlude the goal in the camera frame. In at least one embodiment, using $N_a=4$, $N_e=3$ and saving up to four images per scene, 11.91 million depth images of size 256×256 are rendered on three million scenes. The corresponding camera intrinsic and extrinsic are saved in robot base frame for each depth image.

[0035] FIG. **3**A illustrates a seed trajectory generation pipeline utilizing neural network architecture **300** for generating a plurality of seed trajectories, in accordance with at least one embodiment. The neural network architecture includes the DiffusionSeeder network **100**. In the trajectory generation pipeline **300**, the DiffusionSeeder network **100** includes an observation encoder **310** and a noise prediction network that includes a conditional diffusion model **330**. The observation encoder **310** includes a multi-layer perceptron (MLP) **312** and a vision transformer (ViT) **314**. The ViT **314** takes the depth image **324** and the camera extrinsic **322** (e.g., as output by the MLP **312**) as inputs and generates an environment embedding **332**. A multi-layer perceptron (MLP) **336** embeds the start configuration (e.g., in robot's joint space) and target pose (e.g., of the end effector), as shown in block **318**, into a problem encoding **334**. The problem encoding **334** is concatenated with the environment embedding **332** and then input into the conditional diffusion model **330**. In at least one embodiment, the problem encoding **334** and the environment embedding **332** are concatenated to generate a vector. The conditional diffusion model **330** takes in the concatenated vector for conditioning and a sampled noise **340** (in at least one embodiment, a randomly sampled noise tensor of size 32×7) to generate initial trajectories (in at least one embodiment, the seed trajectories **108** in FIG. **2**B) for an optimizer **350**. The optimizer **350** also takes in the ESDF representation **354** (generated by a functional module **352**, such as NVBLOX) from the depth image **324** for collision checking. The optimizer **350** outputs a final trajectory(ies) **106** after optimization.

[0036] In at least one embodiment, the DiffusionSeeder network **100** uses a conditional Denoising Diffusion Probabilistic Model (DDPM) to generate seed trajectories. The conditional DDPM includes an observation encoder (e.g., the encoder **310** in FIG. **3**A) for environment encoding **332** and/or problem encoding **334**, and a conditional noise prediction network (in at least one embodiment, the conditional diffusion model **330**). The conditional DDPM may be trained on a suitable dataset, such as one generated by the techniques disclosed herein. In at least one embodiment, neural networks configured as conditional DDPMs are utilized. In at least one embodiment, the encoder **310** is configured to provide an environment-related condition to the conditional diffusion model **330**, which iteratively reduces noise through noise prediction based on the environment-related condition provided by the observation encoder. In at least one embodiment, the DiffusionSeeder network **100** (e.g., the conditional DDPM) is combined with an optimizer (e.g., the optimizer **350**), such as CuRobo, to provide diverse seed trajectories (e.g., seed trajectories **108**) for the optimizer **350** to optimize for the final trajectory **106**.

[0037] In at least one embodiment, the encoder **310** takes in depth images **324**, camera poses (e.g., camera extrinsic **322**), start joint configuration (e.g., in block **318**), and goal pose (e.g., in block **318**) as input and encodes the input into a single embedding vector. This embedding may be used to condition the denoising diffusion process (e.g., performed by the conditional diffusion model **330**).

[0038] In at least one embodiment, the VIT **314** has 12 layers and 12 heads and is used as the backbone of the observation encoder network. In at least one embodiment, depth images are represented as H×W×3 by projecting each pixel to a three-dimensional (3D) space using the camera intrinsics. In at least one embodiment, each image is divided into 64×64 patches where each patch is encoded first and concatenated with the linear projection of the SE(3) (Special Euclidean group in 3D) homogeneous transformation of each camera pose. Then, a positional embedding is added to each patch embedding and all the patches are encoded to a single 512-dimensional embedding for visual embedding using the class token of ViT. In at least one embodiment, the visual embedding is concatenated with a 64-dimensional encoding vector of start configuration and goal pose, resulting in a 576-dimensional vector to represent the environment embedding **332**. This vector is then used in the conditional denoising diffusion (e.g., performed by the conditional diffusion model **330**). In at least one embodiment, by utilizing depth observations as input to the observation encoder and a Vision Transformer (ViT) to generate an environment encoding in latent space, motion planning can be accomplished at higher speeds. In at least one embodiment, by generating multiple full trajectories for each inference pass of the diffusion model and avoiding multi-step policy rollouts, the DiffusionSeeder network **100** facilitated an 11× speedup over MπNet and a 57% success rate improvement through collision checking with NVBLOX.

[0039] In at least one embodiment, a CNN-based architecture is adopted for the conditional noise prediction model. In at least one embodiment, the CNN-based architecture adopts a 3-level UNet architecture of channel dimensions [256, 512, 1024] and encodes the time step into a latent vector of 256 dimensions through a MLP **336**, which is concatenated with the environment embedding **332** from the observation encoder **310** as the conditional vector for each layer.

[0040] In at least one embodiment, the observation encoder **310** and noise prediction model (e.g., the condi-

tional diffusion model **330**) are trained jointly. In at least one embodiment, during training, one depth image from the pre-rendered depth images is sampled for each problem. The start and end poses, along with the order of the ground truth trajectories, are reversed half of the time to generate problems that move from the end to the start, enabling further data augmentation. The ground truth trajectories and the input robot start configuration are normalized to be from 0 to 1. At each training iteration, a ground truth trajectory τ is sampled from the dataset. A denoising step $k \in [0, K]$ and a random noise ϵ are randomly selected, which are then added to the ground truth trajectories, resulting in a noisy trajectory $\tilde{\tau}=\tau+\epsilon$. The objective of the noise prediction network (Θ) is to predict the noise added to the original trajectory, with the training loss defined as:

$$L = MSE(\epsilon, \Theta(\tilde{\tau}, k, \Phi(O))), \quad \text{(Eq. 1)}$$

where Φ represents the observation encoder **310**, and O represents the observation.

[0041] In robot motion planning problems, the error in each joint can have different impact on the end effector error, due to the non-linearity of a robot forward kinematics (FK) model. In at least one embodiment, to mitigate this issue, both e and the predicted noise $\hat{\epsilon}$ are passed to the robot FK model to reflect this non-linearity. In addition, as many of the motion planning problems can be solved through a linear interpolation solution, the loss for nonlinear solution may be upweighted to encourage the model to pay attention to the non-linear solutions, which are often harder to solve. In at least one embodiment, the optimizer **350** (such as cuRobo) may be configured to only call a graph-based planner when the linear interpolation solution has failed, which may be used to approximate the non-linearity of the trajectories. Together, in at least one embodiment, the training loss is defined as:

$$\mathcal{L} = (1 + \alpha \mathbb{I}(\tau)) MSE(FK(\epsilon), FK(\Theta(\tilde{\tau}, k, \Phi(O)))), \quad \text{(Eq. 2)}$$

where $\mathbb{I}(\tau)$ is an indicator of whether the ground truth trajectory τ is generated from graph-based planner and α is a scale factor.

[0042] In at least one embodiment, the model is trained with depth images of size 256×256, K=100, α=4 and a batch size of 256 for 72 epochs, which takes six days on eight NVIDIA A100 GPUs.

[0043] In at least one embodiment, a noise of the same shape as the trajectory is randomly sampled during inference time. In at least one embodiment, a Denoising Diffusion Implicit Model (DDIM) is used to generate a denoised trajectory (e.g., a seed trajectory **108**). The present disclosure has recognized that DDIM provides an efficient class of iterative implicit probabilistic models modeling non-Markovian diffusion processes. The training objective of DDIM is the same to that of DDPM, enabling DDIM to be used during inference for faster sampling in the denoising process without re-training the model.

[0044] In at least one embodiment, the trained model is converted to a suitable floating-point format. In at least one embodiment, the trained model is converted to BFLOAT16 (which stands for Brain Floating Point 16-bit) for accelerated inference. In at least one embodiment, the DDIM step is implemented as a fused CUDA kernel leveraging JIT scripting in Pytorch 2.0 and the full inference step is encoded in a single CUDA Graph to further reduce python overheads. In at least one embodiment, incorporating these techniques resulted in a six-millisecond inference time, which included the inference of the vision encoder **310**, followed by five steps of denoising (e.g., by the conditional diffusion model **330**).

[0045] In at least one embodiment, the trained DiffusionSeeder **100** is incorporated with a trajectory optimizer available in CuRobo, and joint space trajectories τ from the DiffusionSeeder network **100**, the robot start configuration q_0, and the target end pose (X_d) are passed to the optimizer **350** to optimize for N_{iters} number of iterations. In at least one embodiment, the trajectory optimization problem is formulated as:

$$\min_{\tau} C_{goal}(X_d, FK(q_T)) + C_{smooth}(\tau) + C_{self}(\tau) + C_{world}(\tau), \quad \text{(Eq. 3)}$$

s.t. joint limits

where C_{goal} is the cost for reaching the goal pose at the final timestep T, C_{smooth} is a cost term to encourage smooth minimum-jerk trajectories, C_{self} and C_{world} are self and world collision avoidance cost terms, respectively.

[0046] In at least one embodiment, in partially observed scenes, for optimizer collision checking, a Euclidean Signed Distance Field (ESDF) (e.g., **354** in FIG. **3**A) is constructed using a suitable module (e.g., the module **352** in FIG. **3**A) from input depth images. In at least one embodiment, NVBLOX, i.e. a GPU-accelerated signed distance field construction library specially designed for robotic path planning, is used. NVBLOX uses parallel computation on the GPU to efficiently perform volumetric mapping of the world, while also enabling sharing of the generated map in a zero-copy mode with cuRobo for trajectory optimization on the GPU.

[0047] FIG. **3**B illustrates a flowchart of a method **360** for motion planning using a neural network architecture for generating a plurality of seed trajectories, in accordance with at least one embodiment. In at least one embodiment, the neural network architecture is embodied as the DiffusionSeeder network **100**. Each block of method **360**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For example, various functions may be carried out by a processor executing instructions stored in memory. The method may also be embodied as computer-usable instructions stored on computer storage media. The method may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, method **360** is described, by way of example, with respect to the system (or framework) of FIGS. **1**A, **1**B, **2**A, **2**B, and **3**A. However, this method may additionally or alternatively be executed by any one system, or any combination of systems, including, but not limited to, those described herein. Furthermore, persons of ordinary skill in the art will understand that any system that performs method **360** is within the scope and spirit of embodiments of the present disclosure.

[0048] At step **362**, the processor generates an environment embedding based on observations of an environment.

[0049] At step **364**, the processor generates a plurality of first trajectories based on random noise and an initial state and a target state corresponding to a robot in the environment. For example, the first trajectories are the noisy trajectories as depicted in FIG. **2**B.

[0050] At step **366**, the processor generates, in parallel, a plurality of second trajectories by denoising the plurality of first trajectories, based on the environment embedding and the plurality of first trajectories. For example, the second trajectories are the seed trajectories as depicted in FIGS. **2**A and **2**B, and/or the initial trajectories as shown in FIG. **3**A.

[0051] At step **368**, the processor determines, based on the plurality of second trajectories, a final trajectory to move the robot in the environment. For example, the processor optimizes the plurality of second trajectories by invoking an optimizer (e.g., **104** in FIG. **1**A or **350** in FIG. **3**A), thereby determining one or more trajectories that can be used for moving the robot in the environment. In another example, the processor obtains optimization trajectories from an optimizer (e.g., **104** in FIG. **1**A or **350** in FIG. **3**A) executed in another processor, and then determines the final trajectory based on the optimization results.

[0052] At step **370**, the processor controls the motion of the robot in the environment according to the final trajectory determined at step **368**. For example, the processor generates control signals to direct the robot's motion based on the final trajectory. Alternatively, the processor can transmit the final trajectory to a driver controller, which then generates control signals.

[0053] FIG. **3**C illustrates a flowchart of a method **380** for training a neural network model for generating a plurality of seed trajectories, in accordance with at least one embodiment. The neural network model implements a neural network architecture provided in the present disclosure. In at least one embodiment, the neural network model is the DiffusionSeeder network **100**. Each block of method **380**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For example, various functions may be carried out by a processor executing instructions stored in memory. The method may also be embodied as computer-usable instructions stored on computer storage media. The method may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, method **380** is described, by way of example, with respect to the system (or framework) of FIGS. **1**A, **1**B, **2**A, **2**B, and **3**A. However, this method may additionally or alternatively be executed by any one system, or any combination of systems, including, but not limited to, those described herein. Furthermore, persons of ordinary skill in the art will understand that any system that performs method **380** is within the scope and spirit of embodiments of the present disclosure.

[0054] At step **382**, the processor samples a ground truth trajectory from a training dataset. The ground truth trajectory corresponds to an environment.

[0055] At step **384**, the processor generates a noisy trajectory based on the ground truth trajectory and a random noise.

[0056] At step **386**, the processor obtains an observation embedding corresponding to the ground truth trajectory. For example, the processor utilizes an encoder (e.g., the observation encoder **210** in FIGS. **2**A and **2**B, or **310** in FIG. **3**A) in the model to generate the observation embedding.

[0057] At step **388**, the processor generates predicted noise based on the observation embedding and a preset denoising step. In at least one embodiment, the processor randomly selects a denoising step for a diffusion pipeline, which is executed by a diffusion model in the model. The denoising step determines the number of iterations for a series of denoising steps in the diffusion pipeline to process a noisy trajectory. The diffusion model performs each denoising step conditioned on the observation embedding from step **386**. After executing the diffusion pipeline, the diffusion model outputs the predicted noise.

[0058] At step **390**, the processor updates learnable weights in the model by minimizing a loss between the random noise (applied at step **384**) and the predicted noise (output from step **388**). For example, the loss function formulated as Equation 1 can be used.

[0059] In at least one embodiment, the training of the model can be performed in various stages. For example, the encoder and the diffusion model in the model can be initially trained separately and then jointly.

Parallel Processing Architecture

[0060] FIG. **4** illustrates a parallel processing unit (PPU) **400**, in accordance with an embodiment. The PPU **400** may be used to implement the seeding motion optimization with diffusion for rapid motion planning. In an embodiment, a processor such as the PPU **400** may be configured to implement a neural network model. The neural network model may be implemented as software instructions executed by the processor or, in other embodiments, the processor can include a matrix of hardware elements configured to process a set of inputs (e.g., electrical signals representing values) to generate a set of outputs, which can represent activations of the neural network model. In yet other embodiments, the neural network model can be implemented as a combination of software instructions and processing performed by a matrix of hardware elements. Implementing the neural network model can include determining a set of parameters for the neural network model through, e.g., supervised or unsupervised training of the neural network model as well as, or in the alternative, performing inference using the set of parameters to process novel sets of inputs.

[0061] In an embodiment, the PPU **400** is a multi-threaded processor that is implemented on one or more integrated circuit devices. The PPU **400** is a latency hiding architecture designed to process many threads in parallel. A thread (e.g., a thread of execution) is an instantiation of a set of instructions configured to be executed by the PPU **400**. In an embodiment, the PPU **400** is a graphics processing unit (GPU) configured to implement a graphics rendering pipeline for processing three-dimensional (3D) graphics data in order to generate two-dimensional (2D) image data for display on a display device. In other embodiments, the PPU **400** may be utilized for performing general-purpose computations. While one exemplary parallel processor is provided herein for illustrative purposes, it should be strongly noted that such processor is set forth for illustrative purposes only, and that any processor may be employed to supplement and/or substitute for the same.

[0062] One or more PPUs **400** may be configured to accelerate thousands of High Performance Computing (HPC), data center, cloud computing, and machine learning applications. The PPU **400** may be configured to accelerate numerous deep learning systems and applications for autonomous vehicles, simulation, computational graphics such as ray or path tracing, deep learning, high-accuracy speech, image, and text recognition systems, intelligent video analytics, molecular simulations, drug discovery, disease diagnosis, weather forecasting, big data analytics, astronomy, molecular dynamics simulation, financial modeling, robotics, factory automation, real-time language translation, online search optimizations, and personalized user recommendations, and the like.

[0063] As shown in FIG. **4**, the PPU **400** includes an Input/Output (I/O) unit **405**, a front end unit **415**, a scheduler unit **420**, a work distribution unit **425**, a hub **430**, a crossbar (Xbar) **470**, one or more general processing clusters (GPCs) **450**, and one or more memory partition units **480**. The PPU **400** may be connected to a host processor or other PPUs **400** via one or more high-speed NVLink **410** interconnect. The PPU **400** may be connected to a host processor or other peripheral devices via an interconnect **402**. The PPU **400** may also be connected to a local memory **404** comprising a number of memory devices. In an embodiment, the local memory may comprise a number of dynamic random access memory (DRAM) devices. The DRAM devices may be configured as a high-bandwidth memory (HBM) subsystem, with multiple DRAM dies stacked within each device.

[0064] The NVLink **410** interconnect enables systems to scale and include one or more PPUs **400** combined with one or more CPUs, supports cache coherence between the PPUs **400** and CPUs, and CPU mastering. Data and/or commands may be transmitted by the NVLink **410** through the hub **430** to/from other units of the PPU **400** such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). The NVLink **410** is described in more detail in conjunction with FIG. **5**B.

[0065] The I/O unit **405** is configured to transmit and receive communications (e.g., commands, data, etc.) from a host processor (not shown) over the interconnect **402**. The I/O unit **405** may communicate with the host processor directly via the interconnect **402** or through one or more intermediate devices such as a memory bridge. In an embodiment, the I/O unit **405** may communicate with one or more other processors, such as one or more the PPUs **400** via the interconnect **402**. In an embodiment, the I/O unit **405** implements a Peripheral Component Interconnect Express (PCIe) interface for communications over a PCIe bus and the interconnect **402** is a PCIe bus. In alternative embodiments, the I/O unit **405** may implement other types of well-known interfaces for communicating with external devices.

[0066] The I/O unit **405** decodes packets received via the interconnect **402**. In an embodiment, the packets represent commands configured to cause the PPU **400** to perform various operations. The I/O unit **405** transmits the decoded commands to various other units of the PPU **400** as the commands may specify. For example, some commands may be transmitted to the front end unit **415**. Other commands may be transmitted to the hub **430** or other units of the PPU **400** such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). In other words, the I/O unit **405** is configured to route communications between and among the various logical units of the PPU **400**.

[0067] In an embodiment, a program executed by the host processor encodes a command stream in a buffer that provides workloads to the PPU **400** for processing. A workload may comprise several instructions and data to be processed by those instructions. The buffer is a region in a memory that is accessible (e.g., read/write) by both the host processor and the PPU **400**. For example, the I/O unit **405** may be configured to access the buffer in a system memory connected to the interconnect **402** via memory requests transmitted over the interconnect **402**. In an embodiment, the host processor writes the command stream to the buffer and then transmits a pointer to the start of the command stream to the PPU **400**. The front end unit **415** receives pointers to one or more command streams. The front end unit **415** manages the one or more streams, reading commands from the streams and forwarding commands to the various units of the PPU **400**.

[0068] The front end unit **415** is coupled to a scheduler unit **420** that configures the various GPCs **450** to process tasks defined by the one or more streams. The scheduler unit **420** is configured to track state information related to the various tasks managed by the scheduler unit **420**. The state may indicate which GPC **450** a task is assigned to, whether the task is active or inactive, a priority level associated with the task, and so forth. The scheduler unit **420** manages the execution of a plurality of tasks on the one or more GPCs **450**.

[0069] The scheduler unit **420** is coupled to a work distribution unit **425** that is configured to dispatch tasks for execution on the GPCs **450**. The work distribution unit **425** may track a number of scheduled tasks received from the scheduler unit **420**. In an embodiment, the work distribution unit **425** manages a pending task pool and an active task pool for each of the GPCs **450**. As a GPC **450** finishes the execution of a task, that task is evicted from the active task pool for the GPC **450** and one of the other tasks from the pending task pool is selected and scheduled for execution on the GPC **450**. If an active task has been idle on the GPC **450**, such as while waiting for a data dependency to be resolved, then the active task may be evicted from the GPC **450** and returned to the pending task pool while another task in the pending task pool is selected and scheduled for execution on the GPC **450**.

[0070] In an embodiment, a host processor executes a driver kernel that implements an application programming interface (API) that enables one or more applications executing on the host processor to schedule operations for execution on the PPU **400**. In an embodiment, multiple compute applications are simultaneously executed by the PPU **400** and the PPU **400** provides isolation, quality of service (QOS), and independent address spaces for the multiple compute applications. An application may generate instructions (e.g., API calls) that cause the driver kernel to generate one or more tasks for execution by the PPU **400**. The driver kernel outputs tasks to one or more streams being processed by the PPU **400**. Each task may comprise one or more groups of related threads, referred to herein as a warp. In an embodiment, a warp comprises 32 related threads that may be executed in parallel. Cooperating threads may refer to a plurality of threads including instructions to perform the task and that may exchange data through shared memory. The

tasks may be allocated to one or more processing units within a GPC **450** and instructions are scheduled for execution by at least one warp.

[0071] The work distribution unit **425** communicates with the one or more GPCs **450** via XBar **470**. The XBar **470** is an interconnect network that couples many of the units of the PPU **400** to other units of the PPU **400**. For example, the XBar **470** may be configured to couple the work distribution unit **425** to a particular GPC **450**. Although not shown explicitly, one or more other units of the PPU **400** may also be connected to the XBar **470** via the hub **430**.

[0072] The tasks are managed by the scheduler unit **420** and dispatched to a GPC **450** by the work distribution unit **425**. The GPC **450** is configured to process the task and generate results. The results may be consumed by other tasks within the GPC **450**, routed to a different GPC **450** via the XBar **470**, or stored in the memory **404**. The results can be written to the memory **404** via the memory partition units **480**, which implement a memory interface for reading and writing data to/from the memory **404**. The results can be transmitted to another PPU **400** or CPU via the NVLink **410**. In an embodiment, the PPU **400** includes a number U of memory partition units **480** that is equal to the number of separate and distinct memory devices of the memory **404** coupled to the PPU **400**. Each GPC **450** may include a memory management unit to provide translation of virtual addresses into physical addresses, memory protection, and arbitration of memory requests. In an embodiment, the memory management unit provides one or more translation lookaside buffers (TLBs) for performing translation of virtual addresses into physical addresses in the memory **404**.

[0073] In an embodiment, the memory partition unit **480** includes a Raster Operations (ROP) unit, a level two (L2) cache, and a memory interface that is coupled to the memory **404**. The memory interface may implement 32, 64, 128, 1024-bit data buses, or the like, for high-speed data transfer. The PPU **400** may be connected to up to Y memory devices, such as high bandwidth memory stacks or graphics double-data-rate, version 5, synchronous dynamic random access memory, or other types of persistent storage. In an embodiment, the memory interface implements an HBM2 memory interface and Y equals half U. In an embodiment, the HBM2 memory stacks are located on the same physical package as the PPU **400**, providing substantial power and area savings compared with conventional GDDR5 SDRAM systems. In an embodiment, each HBM2 stack includes four memory dies and Y equals 4, with each HBM2 stack including two 128-bit channels per die for a total of 8 channels and a data bus width of 1024 bits.

[0074] In an embodiment, the memory **404** supports Single-Error Correcting Double-Error Detecting (SECDED) Error Correction Code (ECC) to protect data. ECC provides higher reliability for compute applications that are sensitive to data corruption. Reliability is especially important in large-scale cluster computing environments where PPUs **400** process very large datasets and/or run applications for extended periods.

[0075] In an embodiment, the PPU **400** implements a multi-level memory hierarchy. In an embodiment, the memory partition unit **480** supports a unified memory to provide a single unified virtual address space for CPU and PPU **400** memory, enabling data sharing between virtual memory systems. In an embodiment the frequency of accesses by a PPU **400** to memory located on other processors is traced to ensure that memory pages are moved to the physical memory of the PPU **400** that is accessing the pages more frequently. In an embodiment, the NVLink **410** supports address translation services allowing the PPU **400** to directly access a CPU's page tables and providing full access to CPU memory by the PPU **400**.

[0076] In an embodiment, copy engines transfer data between multiple PPUs **400** or between PPUs **400** and CPUs. The copy engines can generate page faults for addresses that are not mapped into the page tables. The memory partition unit **480** can then service the page faults, mapping the addresses into the page table, after which the copy engine can perform the transfer. In a conventional system, memory is pinned (e.g., non-pageable) for multiple copy engine operations between multiple processors, substantially reducing the available memory. With hardware page faulting, addresses can be passed to the copy engines without worrying if the memory pages are resident, and the copy process is transparent.

[0077] Data from the memory **404** or other system memory may be fetched by the memory partition unit **480** and stored in the L2 cache **460**, which is located on-chip and is shared between the various GPCs **450**. As shown, each memory partition unit **480** includes a portion of the L2 cache associated with a corresponding memory **404**. Lower level caches may then be implemented in various units within the GPCs **450**. For example, each of the processing units within a GPC **450** may implement a level one (L1) cache. The L1 cache is private memory that is dedicated to a particular processing unit. The L2 cache **460** is coupled to the memory interface **470** and the XBar **470** and data from the L2 cache may be fetched and stored in each of the L1 caches for processing.

[0078] In an embodiment, the processing units within each GPC **450** implement a SIMD (Single-Instruction, Multiple-Data) architecture where each thread in a group of threads (e.g., a warp) is configured to process a different set of data based on the same set of instructions. All threads in the group of threads execute the same instructions. In another embodiment, the processing unit implements a SIMT (Single-Instruction, Multiple Thread) architecture where each thread in a group of threads is configured to process a different set of data based on the same set of instructions, but where individual threads in the group of threads are allowed to diverge during execution. In an embodiment, a program counter, call stack, and execution state is maintained for each warp, enabling concurrency between warps and serial execution within warps when threads within the warp diverge. In another embodiment, a program counter, call stack, and execution state is maintained for each individual thread, enabling equal concurrency between all threads, within and between warps. When execution state is maintained for each individual thread, threads executing the same instructions may be converged and executed in parallel for maximum efficiency.

[0079] Cooperative Groups is a programming model for organizing groups of communicating threads that allows developers to express the granularity at which threads are communicating, enabling the expression of richer, more efficient parallel decompositions. Cooperative launch APIs support synchronization amongst thread blocks for the execution of parallel algorithms. Conventional programming models provide a single, simple construct for synchronizing cooperating threads: a barrier across all threads of a

thread block (e.g., the syncthreads( ) function). However, programmers would often like to define groups of threads at smaller than thread block granularities and synchronize within the defined groups to enable greater performance, design flexibility, and software reuse in the form of collective group-wide function interfaces.

[0080] Cooperative Groups enables programmers to define groups of threads explicitly at sub-block (e.g., as small as a single thread) and multi-block granularities, and to perform collective operations such as synchronization on the threads in a cooperative group. The programming model supports clean composition across software boundaries, so that libraries and utility functions can synchronize safely within their local context without having to make assumptions about convergence. Cooperative Groups primitives enable new patterns of cooperative parallelism, including producer-consumer parallelism, opportunistic parallelism, and global synchronization across an entire grid of thread blocks.

[0081] Each processing unit includes a large number (e.g., 128, etc.) of distinct processing cores (e.g., functional units) that may be fully-pipelined, single-precision, double-precision, and/or mixed precision and include a floating point arithmetic logic unit and an integer arithmetic logic unit. In an embodiment, the floating point arithmetic logic units implement the IEEE 754-2008 standard for floating point arithmetic. In an embodiment, the cores include 64 single-precision (32-bit) floating point cores, 64 integer cores, 32 double-precision (64-bit) floating point cores, and 8 tensor cores.

[0082] Tensor cores configured to perform matrix operations. In particular, the tensor cores are configured to perform deep learning matrix arithmetic, such as GEMM (matrix-matrix multiplication) for convolution operations during neural network training and inferencing. In an embodiment, each tensor core operates on a 4×4 matrix and performs a matrix multiply and accumulate operation D=A×B+C, where A, B, C, and D are 4×4 matrices.

[0083] In an embodiment, the matrix multiply inputs A and B may be integer, fixed-point, or floating point matrices, while the accumulation matrices C and D may be integer, fixed-point, or floating point matrices of equal or higher bitwidths. In an embodiment, tensor cores operate on one, four, or eight bit integer input data with 32-bit integer accumulation. The 8-bit integer matrix multiply requires 1024 operations and results in a full precision product that is then accumulated using 32-bit integer addition with the other intermediate products for a 8×8×16 matrix multiply. In an embodiment, tensor Cores operate on 16-bit floating point input data with 32-bit floating point accumulation. The 16-bit floating point multiply requires 64 operations and results in a full precision product that is then accumulated using 32-bit floating point addition with the other intermediate products for a 4×4×4 matrix multiply. In practice, Tensor Cores are used to perform much larger two-dimensional or higher dimensional matrix operations, built up from these smaller elements. An API, such as CUDA 9 C++ API, exposes specialized matrix load, matrix multiply and accumulate, and matrix store operations to efficiently use Tensor Cores from a CUDA-C++ program. At the CUDA level, the warp-level interface assumes 16×16 size matrices spanning all 32 threads of the warp.

[0084] Each processing unit may also comprise M special function units (SFUs) that perform special functions (e.g., attribute evaluation, reciprocal square root, and the like). In an embodiment, the SFUs may include a tree traversal unit configured to traverse a hierarchical tree data structure. In an embodiment, the SFUs may include texture unit configured to perform texture map filtering operations. In an embodiment, the texture units are configured to load texture maps (e.g., a 2D array of texels) from the memory **404** and sample the texture maps to produce sampled texture values for use in shader programs executed by the processing unit. In an embodiment, the texture maps are stored in shared memory that may comprise or include an L1 cache. The texture units implement texture operations such as filtering operations using mip-maps (e.g., texture maps of varying levels of detail). In an embodiment, each processing unit includes two texture units.

[0085] Each processing unit also comprises N load store units (LSUs) that implement load and store operations between the shared memory and the register file. Each processing unit includes an interconnect network that connects each of the cores to the register file and the LSU to the register file, shared memory. In an embodiment, the interconnect network is a crossbar that can be configured to connect any of the cores to any of the registers in the register file and connect the LSUs to the register file and memory locations in shared memory.

[0086] The shared memory is an array of on-chip memory that allows for data storage and communication between the processing units and between threads within a processing unit. In an embodiment, the shared memory comprises 128 KB of storage capacity and is in the path from each of the processing units to the memory partition unit **480**. The shared memory can be used to cache reads and writes. One or more of the shared memory, L1 cache, L2 cache, and memory **404** are backing stores.

[0087] Combining data cache and shared memory functionality into a single memory block provides the best overall performance for both types of memory accesses. The capacity is usable as a cache by programs that do not use shared memory. For example, if shared memory is configured to use half of the capacity, texture and load/store operations can use the remaining capacity. Integration within the shared memory enables the shared memory to function as a high-throughput conduit for streaming data while simultaneously providing high-bandwidth and low-latency access to frequently reused data.

[0088] When configured for general purpose parallel computation, a simpler configuration can be used compared with graphics processing. Specifically, fixed function graphics processing units, are bypassed, creating a much simpler programming model. In the general purpose parallel computation configuration, the work distribution unit **425** assigns and distributes blocks of threads directly to the processing units within the GPCs **450**. Threads execute the same program, using a unique thread ID in the calculation to ensure each thread generates unique results, using the processing unit(s) to execute the program and perform calculations, shared memory to communicate between threads, and the LSU to read and write global memory through the shared memory and the memory partition unit **480**. When configured for general purpose parallel computation, the processing units can also write commands that the scheduler unit **420** can use to launch new work on the processing units.

[0089] The PPUs **400** may each include, and/or be configured to perform functions of, one or more processing

cores and/or components thereof, such as Tensor Cores (TCs), Tensor Processing Units (TPUs), Pixel Visual Cores (PVCs), Ray Tracing (RT) Cores, Vision Processing Units (VPUs), Graphics Processing Clusters (GPCs), Texture Processing Clusters (TPCs), Streaming Multiprocessors (SMs), Tree Traversal Units (TTUs), Artificial Intelligence Accelerators (AIAs), Deep Learning Accelerators (DLAs), Arithmetic-Logic Units (ALUs), Application-Specific Integrated Circuits (ASICs), Floating Point Units (FPUs), input/output (I/O) elements, peripheral component interconnect (PCI) or peripheral component interconnect express (PCIe) elements, and/or the like.

[0090] The PPU **400** may be included in a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, and the like. In an embodiment, the PPU **400** is embodied on a single semiconductor substrate. In another embodiment, the PPU **400** is included in a system-on-a-chip (SoC) along with one or more other devices such as additional PPUs **400**, the memory **404**, a reduced instruction set computer (RISC) CPU, a memory management unit (MMU), a digital-to-analog converter (DAC), and the like.

[0091] In an embodiment, the PPU **400** may be included on a graphics card that includes one or more memory devices. The graphics card may be configured to interface with a PCIe slot on a motherboard of a desktop computer. In yet another embodiment, the PPU **400** may be an integrated graphics processing unit (iGPU) or parallel processor included in the chipset of the motherboard. In yet another embodiment, the PPU **400** may be realized in reconfigurable hardware. In yet another embodiment, parts of the PPU **400** may be realized in reconfigurable hardware.

Exemplary Computing System

[0092] Systems with multiple GPUs and CPUs are used in a variety of industries as developers expose and leverage more parallelism in applications such as artificial intelligence computing. High-performance GPU-accelerated systems with tens to many thousands of compute nodes are deployed in data centers, research facilities, and supercomputers to solve ever larger problems. As the number of processing devices within the high-performance systems increases, the communication and data transfer mechanisms need to scale to support the increased bandwidth.

[0093] FIG. **5**A is a conceptual diagram of a processing system **500** implemented using the PPU **400** of FIG. **4**, in accordance with an embodiment. The exemplary system **500** may be configured to implement seeding motion optimization with diffusion for rapid motion planning. The processing system **500** includes a CPU **530**, switch **510**, and multiple PPUs **400**, and respective memories **404**.

[0094] The NVLink **410** provides high-speed communication links between each of the PPUs **400**. Although a particular number of NVLink **410** and interconnect **402** connections are illustrated in FIG. **5**B, the number of connections to each PPU **400** and the CPU **530** may vary. The switch **510** interfaces between the interconnect **402** and the CPU **530**. The PPUs **400**, memories **404**, and NVLinks **410** may be situated on a single semiconductor platform to form a parallel processing module **525**. In an embodiment, the switch **510** supports two or more protocols to interface between various different connections and/or links.

[0095] In another embodiment (not shown), the NVLink **410** provides one or more high-speed communication links between each of the PPUs **400** and the CPU **530** and the switch **510** interfaces between the interconnect **402** and each of the PPUs **400**. The PPUs **400**, memories **404**, and interconnect **402** may be situated on a single semiconductor platform to form a parallel processing module **525**. In yet another embodiment (not shown), the interconnect **402** provides one or more communication links between each of the PPUs **400** and the CPU **530** and the switch **510** interfaces between each of the PPUs **400** using the NVLink **410** to provide one or more high-speed communication links between the PPUs **400**. In another embodiment (not shown), the NVLink **410** provides one or more high-speed communication links between the PPUs **400** and the CPU **530** through the switch **510**. In yet another embodiment (not shown), the interconnect **402** provides one or more communication links between each of the PPUs **400** directly. One or more of the NVLink **410** high-speed communication links may be implemented as a physical NVLink interconnect or either an on-chip or on-die interconnect using the same protocol as the NVLink **410**.

[0096] In the context of the present description, a single semiconductor platform may refer to a sole unitary semiconductor-based integrated circuit fabricated on a die or chip. It should be noted that the term single semiconductor platform may also refer to multi-chip modules with increased connectivity which simulate on-chip operation and make substantial improvements over utilizing a conventional bus implementation. Of course, the various circuits or devices may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. Alternately, the parallel processing module **525** may be implemented as a circuit board substrate and each of the PPUs **400** and/or memories **404** may be packaged devices. In an embodiment, the CPU **530**, switch **510**, and the parallel processing module **525** are situated on a single semiconductor platform.

[0097] In an embodiment, the signaling rate of each NVLink **410** is 20 to 25 Gigabits/second and each PPU **400** includes six NVLink **410** interfaces (as shown in FIG. **5**A, five NVLink **410** interfaces are included for each PPU **400**). Each NVLink **410** provides a data transfer rate of 25 Gigabytes/second in each direction, with six links providing 400 Gigabytes/second. The NVLinks **410** can be used exclusively for PPU-to-PPU communication as shown in FIG. **5**A, or some combination of PPU-to-PPU and PPU-to-CPU, when the CPU **530** also includes one or more NVLink **410** interfaces.

[0098] In an embodiment, the NVLink **410** allows direct load/store/atomic access from the CPU **530** to each PPU's **400** memory **404**. In an embodiment, the NVLink **410** supports coherency operations, allowing data read from the memories **404** to be stored in the cache hierarchy of the CPU **530**, reducing cache access latency for the CPU **530**. In an embodiment, the NVLink **410** includes support for Address Translation Services (ATS), allowing the PPU **400** to directly access page tables within the CPU **530**. One or more of the NVLinks **410** may also be configured to operate in a low-power mode.

[0099] FIG. **5**B illustrates an exemplary system **565** in which the various architecture and/or functionality of the various previous embodiments may be implemented. The

exemplary system **565** may be configured to implement seeding motion optimization with diffusion for rapid motion planning.

[0100] As shown, a system **565** is provided including at least one central processing unit **530** that is connected to a communication bus **575**. The communication bus **575** may directly or indirectly couple one or more of the following devices: main memory **540**, network interface **535**, CPU(s) **530**, display device(s) **545**, input device(s) **560**, switch **510**, and parallel processing system **525**. The communication bus **575** may be implemented using any suitable protocol and may represent one or more links or busses, such as an address bus, a data bus, a control bus, or a combination thereof. The communication bus **575** may include one or more bus or link types, such as an industry standard architecture (ISA) bus, an extended industry standard architecture (EISA) bus, a video electronics standards association (VESA) bus, a peripheral component interconnect (PCI) bus, a peripheral component interconnect express (PCIe) bus, HyperTransport, and/or another type of bus or link. In some embodiments, there are direct connections between components. As an example, the CPU(s) **530** may be directly connected to the main memory **540**. Further, the CPU(s) **530** may be directly connected to the parallel processing system **525**. Where there is direct, or point-to-point connection between components, the communication bus **575** may include a PCIe link to carry out the connection. In these examples, a PCI bus need not be included in the system **565**.

[0101] Although the various blocks of FIG. **5**B are shown as connected via the communication bus **575** with lines, this is not intended to be limiting and is for clarity only. For example, in some embodiments, a presentation component, such as display device(s) **545**, may be considered an I/O component, such as input device(s) **560** (e.g., if the display is a touch screen). As another example, the CPU(s) **530** and/or parallel processing system **525** may include memory (e.g., the main memory **540** may be representative of a storage device in addition to the parallel processing system **525**, the CPUs **530**, and/or other components). In other words, the computing device of FIG. **5**B is merely illustrative. Distinction is not made between such categories as "workstation," "server," "laptop," "desktop," "tablet," "client device," "mobile device," "hand-held device," "game console," "electronic control unit (ECU)," "virtual reality system," and/or other device or system types, as all are contemplated within the scope of the computing device of FIG. **5**B.

[0102] The system **565** also includes a main memory **540**. Control logic (software) and data are stored in the main memory **540** which may take the form of a variety of computer-readable media. The computer-readable media may be any available media that may be accessed by the system **565**. The computer-readable media may include both volatile and nonvolatile media, and removable and non-removable media. By way of example, and not limitation, the computer-readable media may comprise computer-storage media and communication media.

[0103] The computer-storage media may include both volatile and nonvolatile media and/or removable and non-removable media implemented in any method or technology for storage of information such as computer-readable instructions, data structures, program modules, and/or other data types. For example, the main memory **540** may store computer-readable instructions (e.g., that represent a program(s) and/or a program element(s), such as an operating system. Computer-storage media may include, but is not limited to, RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium which may be used to store the desired information and which may be accessed by system **565**. As used herein, computer storage media does not comprise signals per se.

[0104] The computer storage media may embody computer-readable instructions, data structures, program modules, and/or other data types in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. The term "modulated data signal" may refer to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal. By way of example, and not limitation, the computer storage media may include wired media such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media. Combinations of any of the above should also be included within the scope of computer-readable media.

[0105] Computer programs, when executed, enable the system **565** to perform various functions. The CPU(s) **530** may be configured to execute at least some of the computer-readable instructions to control one or more components of the system **565** to perform one or more of the methods and/or processes described herein. The CPU(s) **530** may each include one or more cores (e.g., one, two, four, eight, twenty-eight, seventy-two, etc.) that are capable of handling a multitude of software threads simultaneously. The CPU(s) **530** may include any type of processor, and may include different types of processors depending on the type of system **565** implemented (e.g., processors with fewer cores for mobile devices and processors with more cores for servers). For example, depending on the type of system **565**, the processor may be an Advanced RISC Machines (ARM) processor implemented using Reduced Instruction Set Computing (RISC) or an x86 processor implemented using Complex Instruction Set Computing (CISC). The system **565** may include one or more CPUs **530** in addition to one or more microprocessors or supplementary co-processors, such as math co-processors.

[0106] In addition to or alternatively from the CPU(s) **530**, the parallel processing module **525** may be configured to execute at least some of the computer-readable instructions to control one or more components of the system **565** to perform one or more of the methods and/or processes described herein. The parallel processing module **525** may be used by the system **565** to render graphics (e.g., 3D graphics) or perform general purpose computations. For example, the parallel processing module **525** may be used for General-Purpose computing on GPUs (GPGPU). In embodiments, the CPU(s) **530** and/or the parallel processing module **525** may discretely or jointly perform any combination of the methods, processes and/or portions thereof.

[0107] The system **565** also includes input device(s) **560**, the parallel processing system **525**, and display device(s) **545**. The display device(s) **545** may include a display (e.g., a monitor, a touch screen, a television screen, a heads-up-display (HUD), other display types, or a combination thereof), speakers, and/or other presentation components.

The display device(s) **545** may receive data from other components (e.g., the parallel processing system **525**, the CPU(s) **530**, etc.), and output the data (e.g., as an image, video, sound, etc.).

[0108] The network interface **535** may enable the system **565** to be logically coupled to other devices including the input devices **560**, the display device(s) **545**, and/or other components, some of which may be built in to (e.g., integrated in) the system **565**. Illustrative input devices **560** include a microphone, mouse, keyboard, joystick, game pad, game controller, satellite dish, scanner, printer, wireless device, etc. The input devices **560** may provide a natural user interface (NUI) that processes air gestures, voice, or other physiological inputs generated by a user. In some instances, inputs may be transmitted to an appropriate network element for further processing. An NUI may implement any combination of speech recognition, stylus recognition, facial recognition, biometric recognition, gesture recognition both on screen and adjacent to the screen, air gestures, head and eye tracking, and touch recognition (as described in more detail below) associated with a display of the system **565**. The system **565** may be include depth cameras, such as stereoscopic camera systems, infrared camera systems, RGB camera systems, touchscreen technology, and combinations of these, for gesture detection and recognition. Additionally, the system **565** may include accelerometers or gyroscopes (e.g., as part of an inertia measurement unit (IMU)) that enable detection of motion. In some examples, the output of the accelerometers or gyroscopes may be used by the system **565** to render immersive augmented reality or virtual reality.

[0109] Further, the system **565** may be coupled to a network (e.g., a telecommunications network, local area network (LAN), wireless network, wide area network (WAN) such as the Internet, peer-to-peer network, cable network, or the like) through a network interface **535** for communication purposes. The system **565** may be included within a distributed network and/or cloud computing environment.

[0110] The network interface **535** may include one or more receivers, transmitters, and/or transceivers that enable the system **565** to communicate with other computing devices via an electronic communication network, included wired and/or wireless communications. The network interface **535** may be implemented as a network interface controller (NIC) that includes one or more data processing units (DPUs) to perform operations such as (for example and without limitation) packet parsing and accelerating network processing and communication. The network interface **535** may include components and functionality to enable communication over any of a number of different networks, such as wireless networks (e.g., Wi-Fi, Z-Wave, Bluetooth, Bluetooth LE, ZigBee, etc.), wired networks (e.g., communicating over Ethernet or InfiniBand), low-power wide-area networks (e.g., LoRaWAN, SigFox, etc.), and/or the Internet.

[0111] The system **565** may also include a secondary storage (not shown). The secondary storage includes, for example, a hard disk drive and/or a removable storage drive, representing a floppy disk drive, a magnetic tape drive, a compact disk drive, digital versatile disk (DVD) drive, recording device, universal serial bus (USB) flash memory. The removable storage drive reads from and/or writes to a removable storage unit in a well-known manner. The system **565** may also include a hard-wired power supply, a battery power supply, or a combination thereof (not shown). The power supply may provide power to the system **565** to enable the components of the system **565** to operate.

[0112] Each of the foregoing modules and/or devices may even be situated on a single semiconductor platform to form the system **565**. Alternately, the various modules may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. While various embodiments have been described above, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

Example Network Environments

[0113] Network environments suitable for use in implementing embodiments of the disclosure may include one or more client devices, servers, network attached storage (NAS), other backend devices, and/or other device types. The client devices, servers, and/or other device types (e.g., each device) may be implemented on one or more instances of the processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B—e.g., each device may include similar components, features, and/or functionality of the processing system **500** and/or exemplary system **565**.

[0114] Components of a network environment may communicate with each other via a network(s), which may be wired, wireless, or both. The network may include multiple networks, or a network of networks. By way of example, the network may include one or more Wide Area Networks (WANs), one or more Local Area Networks (LANs), one or more public networks such as the Internet and/or a public switched telephone network (PSTN), and/or one or more private networks. Where the network includes a wireless telecommunications network, components such as a base station, a communications tower, or even access points (as well as other components) may provide wireless connectivity.

[0115] Compatible network environments may include one or more peer-to-peer network environments—in which case a server may not be included in a network environment—and one or more client-server network environments—in which case one or more servers may be included in a network environment. In peer-to-peer network environments, functionality described herein with respect to a server(s) may be implemented on any number of client devices.

[0116] In at least one embodiment, a network environment may include one or more cloud-based network environments, a distributed computing environment, a combination thereof, etc. A cloud-based network environment may include a framework layer, a job scheduler, a resource manager, and a distributed file system implemented on one or more of servers, which may include one or more core network servers and/or edge servers. A framework layer may include a framework to support software of a software layer and/or one or more application(s) of an application layer. The software or application(s) may respectively include web-based service software or applications. In embodiments, one or more of the client devices may use the web-based service software or applications (e.g., by access-

ing the service software and/or applications via one or more application programming interfaces (APIs)). The framework layer may be, but is not limited to, a type of free and open-source software web application framework such as that may use a distributed file system for large-scale data processing (e.g., "big data").

[0117] A cloud-based network environment may provide cloud computing and/or cloud storage that carries out any combination of computing and/or data storage functions described herein (or one or more portions thereof). Any of these various functions may be distributed over multiple locations from central or core servers (e.g., of one or more data centers that may be distributed across a state, a region, a country, the globe, etc.). If a connection to a user (e.g., a client device) is relatively close to an edge server(s), a core server(s) may designate at least a portion of the functionality to the edge server(s). A cloud-based network environment may be private (e.g., limited to a single organization), may be public (e.g., available to many organizations), and/or a combination thereof (e.g., a hybrid cloud environment).

[0118] The client device(s) may include at least some of the components, features, and functionality of the example processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B. By way of example and not limitation, a client device may be embodied as a Personal Computer (PC), a laptop computer, a mobile device, a smartphone, a tablet computer, a smart watch, a wearable computer, a Personal Digital Assistant (PDA), an MP3 player, a virtual reality headset, a Global Positioning System (GPS) or device, a video player, a video camera, a surveillance device or system, a vehicle, a boat, a flying vessel, a virtual machine, a drone, a robot, a handheld communications device, a hospital device, a gaming device or system, an entertainment system, a vehicle computer system, an embedded system controller, a remote control, an appliance, a consumer electronic device, a workstation, an edge device, any combination of these delineated devices, or any other suitable device.

Machine Learning

[0119] Deep neural networks (DNNs) developed on processors, such as the PPU **400** have been used for diverse use cases, from self-driving cars to faster drug development, from automatic image captioning in online image databases to smart real-time language translation in video chat applications. Deep learning is a technique that models the neural learning process of the human brain, continually learning, continually getting smarter, and delivering more accurate results more quickly over time. A child is initially taught by an adult to correctly identify and classify various shapes, eventually being able to identify shapes without any coaching. Similarly, a deep learning or neural learning system needs to be trained in object recognition and classification for it get smarter and more efficient at identifying basic objects, occluded objects, etc., while also assigning context to objects.

[0120] At the simplest level, neurons in the human brain look at various inputs that are received, importance levels are assigned to each of these inputs, and output is passed on to other neurons to act upon. An artificial neuron or perceptron is the most basic model of a neural network. In one example, a perceptron may receive one or more inputs that represent various features of an object that the perceptron is being trained to recognize and classify, and each of these features is assigned a certain weight based on the importance of that feature in defining the shape of an object.

[0121] A deep neural network (DNN) model includes multiple layers of many connected nodes (e.g., perceptrons, Boltzmann machines, radial basis functions, convolutional layers, etc.) that can be trained with enormous amounts of input data to quickly solve complex problems with high accuracy. In one example, a first layer of the DNN model breaks down an input image of an automobile into various sections and looks for basic patterns such as lines and angles. The second layer assembles the lines to look for higher level patterns such as wheels, windshields, and mirrors. The next layer identifies the type of vehicle, and the final few layers generate a label for the input image, identifying the model of a specific automobile brand.

[0122] Once the DNN is trained, the DNN can be deployed and used to identify and classify objects or patterns in a process known as inference. Examples of inference (the process through which a DNN extracts useful information from a given input) include identifying handwritten numbers on checks deposited into ATM machines, identifying images of friends in photos, delivering movie recommendations to over fifty million users, identifying and classifying different types of automobiles, pedestrians, and road hazards in driverless cars, or translating human speech in real-time.

[0123] During training, data flows through the DNN in a forward propagation phase until a prediction is produced that indicates a label corresponding to the input. If the neural network does not correctly label the input, then errors between the correct label and the predicted label are analyzed, and the weights are adjusted for each feature during a backward propagation phase until the DNN correctly labels the input and other inputs in a training dataset. Training complex neural networks requires massive amounts of parallel computing performance, including floating-point multiplications and additions that are supported by the PPU **400**. Inferencing is less compute-intensive than training, being a latency-sensitive process where a trained neural network is applied to new inputs it has not seen before to classify images, detect emotions, identify recommendations, recognize and translate speech, and generally infer new information.

[0124] Neural networks rely heavily on matrix math operations, and complex multi-layered networks require tremendous amounts of floating-point performance and bandwidth for both efficiency and speed. With thousands of processing cores, optimized for matrix math operations, and delivering tens to hundreds of TFLOPS of performance, the PPU **400** is a computing platform capable of delivering performance required for deep neural network-based artificial intelligence and machine learning applications.

[0125] Furthermore, images generated applying one or more of the techniques disclosed herein may be used to train, test, or certify DNNs used to recognize objects and environments in the real world. Such images may include scenes of roadways, factories, buildings, urban settings, rural settings, humans, animals, and any other physical object or real-world setting. Such images may be used to train, test, or certify DNNs that are employed in machines or robots to manipulate, handle, or modify physical objects in the real world. Furthermore, such images may be used to train, test, or certify DNNs that are employed in autonomous vehicles to navigate and move the vehicles through the real world. Additionally, images generated applying one or more of the

techniques disclosed herein may be used to convey information to users of such machines, robots, and vehicles.

[0126] FIG. **5**C illustrates components of an exemplary system **555** that can be used to train and utilize machine learning, in accordance with at least one embodiment. As will be discussed, various components can be provided by various combinations of computing devices and resources, or a single computing system, which may be under control of a single entity or multiple entities. Further, aspects may be triggered, initiated, or requested by different entities. In at least one embodiment training of a neural network might be instructed by a provider associated with provider environment **506**, while in at least one embodiment training might be requested by a customer or other user having access to a provider environment through a client device **502** or other such resource. In at least one embodiment, training data (or data to be analyzed by a trained neural network) can be provided by a provider, a user, or a third party content provider **524**. In at least one embodiment, client device **502** may be a vehicle or object that is to be navigated on behalf of a user, for example, which can submit requests and/or receive instructions that assist in navigation of a device.

[0127] In at least one embodiment, requests are able to be submitted across at least one network **504** to be received by a provider environment **506**. In at least one embodiment, a client device may be any appropriate electronic and/or computing devices enabling a user to generate and send such requests, such as, but not limited to, desktop computers, notebook computers, computer servers, smartphones, tablet computers, gaming consoles (portable or otherwise), computer processors, computing logic, and set-top boxes. Network(s) **504** can include any appropriate network for transmitting a request or other such data, as may include Internet, an intranet, an Ethernet, a cellular network, a local area network (LAN), a wide area network (WAN), a personal area network (PAN), an ad hoc network of direct wireless connections among peers, and so on.

[0128] In at least one embodiment, requests can be received at an interface layer **508**, which can forward data to a training and inference manager **532**, in this example. The training and inference manager **532** can be a system or service including hardware and software for managing requests and service corresponding data or content, in at least one embodiment, the training and inference manager **532** can receive a request to train a neural network, and can provide data for a request to a training module **512**. In at least one embodiment, training module **512** can select an appropriate model or neural network to be used, if not specified by the request, and can train a model using relevant training data. In at least one embodiment, training data can be a batch of data stored in a training data repository **514**, received from client device **502**, or obtained from a third party provider **524**. In at least one embodiment, training module **512** can be responsible for training data. A neural network can be any appropriate network, such as a recurrent neural network (RNN) or convolutional neural network (CNN). Once a neural network is trained and successfully evaluated, a trained neural network can be stored in a model repository **516**, for example, that may store different models or networks for users, applications, or services, etc. In at least one embodiment, there may be multiple models for a single application or entity, as may be utilized based on a number of different factors.

[0129] In at least one embodiment, at a subsequent point in time, a request may be received from client device **502** (or another such device) for content (e.g., path determinations) or data that is at least partially determined or impacted by a trained neural network. This request can include, for example, input data to be processed using a neural network to obtain one or more inferences or other output values, classifications, or predictions, or for at least one embodiment, input data can be received by interface layer **508** and directed to inference module **518**, although a different system or service can be used as well. In at least one embodiment, inference module **518** can obtain an appropriate trained network, such as a trained deep neural network (DNN) as discussed herein, from model repository **516** if not already stored locally to inference module **518**. Inference module **518** can provide data as input to a trained network, which can then generate one or more inferences as output. This may include, for example, a classification of an instance of input data. In at least one embodiment, inferences can then be transmitted to client device **502** for display or other communication to a user. In at least one embodiment, context data for a user may also be stored to a user context data repository **522**, which may include data about a user which may be useful as input to a network in generating inferences, or determining data to return to a user after obtaining instances. In at least one embodiment, relevant data, which may include at least some of input or inference data, may also be stored to a local database **534** for processing future requests. In at least one embodiment, a user can use account information or other information to access resources or functionality of a provider environment. In at least one embodiment, if permitted and available, user data may also be collected and used to further train models, in order to provide more accurate inferences for future requests. In at least one embodiment, requests may be received through a user interface to a machine learning application **526** executing on client device **502**, and results displayed through a same interface. A client device can include resources such as a processor **528** and memory **562** for generating a request and processing results or a response, as well as at least one data storage element **552** for storing data for machine learning application **526**.

[0130] In at least one embodiment a processor **528** (or a processor of training module **512** or inference module **518**) will be a central processing unit (CPU). As mentioned, however, resources in such environments can utilize GPUs to process data for at least certain types of requests. With thousands of cores, GPUs, such as PPU **400** are designed to handle substantial parallel workloads and, therefore, have become popular in deep learning for training neural networks and generating predictions. While use of GPUs for offline builds has enabled faster training of larger and more complex models, generating predictions offline implies that either request-time input features cannot be used or predictions must be generated for all permutations of features and stored in a lookup table to serve real-time requests. If a deep learning framework supports a CPU-mode and a model is small and simple enough to perform a feed-forward on a CPU with a reasonable latency, then a service on a CPU instance could host a model. In this case, training can be done offline on a GPU and inference done in real-time on a CPU. If a CPU approach is not viable, then a service can run on a GPU instance. Because GPUs have different performance and cost characteristics than CPUs, however, running

a service that offloads a runtime algorithm to a GPU can require it to be designed differently from a CPU based service.

[0131] In at least one embodiment, video data can be provided from client device **502** for enhancement in provider environment **506**. In at least one embodiment, video data can be processed for enhancement on client device **502**. In at least one embodiment, video data may be streamed from a third party content provider **524** and enhanced by third party content provider **524**, provider environment **506**, or client device **502**. In at least one embodiment, video data can be provided from client device **502** for use as training data in provider environment **506**.

[0132] In at least one embodiment, supervised and/or unsupervised training can be performed by the client device **502** and/or the provider environment **506**. In at least one embodiment, a set of training data **514** (e.g., classified or labeled data) is provided as input to function as training data. In at least one embodiment, training data can include instances of at least one type of object for which a neural network is to be trained, as well as information that identifies that type of object. In at least one embodiment, training data might include a set of images that each includes a representation of a type of object, where each image also includes, or is associated with, a label, metadata, classification, or other piece of information identifying a type of object represented in a respective image. Various other types of data may be used as training data as well, as may include text data, audio data, video data, and so on. In at least one embodiment, training data **514** is provided as training input to a training module **512**. In at least one embodiment, training module **512** can be a system or service that includes hardware and software, such as one or more computing devices executing a training application, for training a neural network (or other model or algorithm, etc.). In at least one embodiment, training module **512** receives an instruction or request indicating a type of model to be used for training, in at least one embodiment, a model can be any appropriate statistical model, network, or algorithm useful for such purposes, as may include an artificial neural network, deep learning algorithm, learning classifier, Bayesian network, and so on. In at least one embodiment, training module **512** can select an initial model, or other untrained model, from an appropriate repository **516** and utilize training data **514** to train a model, thereby generating a trained model (e.g., trained deep neural network) that can be used to classify similar types of data, or generate other such inferences. In at least one embodiment where training data is not used, an appropriate initial model can still be selected for training on input data per training module **512**.

[0133] In at least one embodiment, a model can be trained in a number of different ways, as may depend in part upon a type of model selected. In at least one embodiment, a machine learning algorithm can be provided with a set of training data, where a model is a model artifact created by a training process. In at least one embodiment, each instance of training data contains a correct answer (e.g., classification), which can be referred to as a target or target attribute. In at least one embodiment, a learning algorithm finds patterns in training data that map input data attributes to a target, an answer to be predicted, and a machine learning model is output that captures these patterns. In at least one embodiment, a machine learning model can then be used to obtain predictions on new data for which a target is not specified.

[0134] In at least one embodiment, training and inference manager **532** can select from a set of machine learning models including binary classification, multiclass classification, generative, and regression models. In at least one embodiment, a type of model to be used can depend at least in part upon a type of target to be predicted.

Graphics Processing Pipeline

[0135] In an embodiment, the PPU **400** comprises a graphics processing unit (GPU). The PPU **400** is configured to receive commands that specify shader programs for processing graphics data. Graphics data may be defined as a set of primitives such as points, lines, triangles, quads, triangle strips, and the like. Typically, a primitive includes data that specifies a number of vertices for the primitive (e.g., in a model-space coordinate system) as well as attributes associated with each vertex of the primitive. The PPU **400** can be configured to process the graphics primitives to generate a frame buffer (e.g., pixel data for each of the pixels of the display).

[0136] An application writes model data for a scene (e.g., a collection of vertices and attributes) to a memory such as a system memory or memory **404**. The model data defines each of the objects that may be visible on a display. The application then makes an API call to the driver kernel that requests the model data to be rendered and displayed. The driver kernel reads the model data and writes commands to the one or more streams to perform operations to process the model data. The commands may reference different shader programs to be implemented on the processing units within the PPU **400** including one or more of a vertex shader, hull shader, domain shader, geometry shader, and a pixel shader. For example, one or more of the processing units may be configured to execute a vertex shader program that processes a number of vertices defined by the model data. In an embodiment, the different processing units may be configured to execute different shader programs concurrently. For example, a first subset of processing units may be configured to execute a vertex shader program while a second subset of processing units may be configured to execute a pixel shader program. The first subset of processing units processes vertex data to produce processed vertex data and writes the processed vertex data to the L2 cache **460** and/or the memory **404**. After the processed vertex data is rasterized (e.g., transformed from three-dimensional data into two-dimensional data in screen space) to produce fragment data, the second subset of processing units executes a pixel shader to produce processed fragment data, which is then blended with other processed fragment data and written to the frame buffer in memory **404**. The vertex shader program and pixel shader program may execute concurrently, processing different data from the same scene in a pipelined fashion until all of the model data for the scene has been rendered to the frame buffer. Then, the contents of the frame buffer are transmitted to a display controller for display on a display device.

[0137] A graphics processing pipeline may be implemented via an application executed by a host processor, such as a CPU. In an embodiment, a device driver may implement an application programming interface (API) that defines various functions that can be utilized by an application in

order to generate graphical data for display. The device driver is a software program that includes a plurality of instructions that control the operation of the PPU **400**. The API provides an abstraction for a programmer that lets a programmer utilize specialized graphics hardware, such as the PPU **400**, to generate the graphical data without requiring the programmer to utilize the specific instruction set for the PPU **400**. The application may include an API call that is routed to the device driver for the PPU **400**. The device driver interprets the API call and performs various operations to respond to the API call. In some instances, the device driver may perform operations by executing instructions on the CPU. In other instances, the device driver may perform operations, at least in part, by launching operations on the PPU **400** utilizing an input/output interface between the CPU and the PPU **400**. In an embodiment, the device driver is configured to implement the graphics processing pipeline utilizing the hardware of the PPU **400**.

[0138] Various programs may be executed within the PPU **400** in order to implement the various stages of the graphics processing pipeline. For example, the device driver may launch a kernel on the PPU **400** to perform a vertex shading stage on one processing unit (or multiple processing units). The device driver (or the initial kernel executed by the PPU **400**) may also launch other kernels on the PPU **400** to perform other stages of the graphics processing pipeline, such as a geometry shading stage and a fragment shading stage. In addition, some of the stages of the graphics processing pipeline may be implemented on fixed unit hardware such as a rasterizer or a data assembler implemented within the PPU **400**. It will be appreciated that results from one kernel may be processed by one or more intervening fixed function hardware units before being processed by a subsequent kernel on a processing unit.

[0139] Images generated applying one or more of the techniques disclosed herein may be displayed on a monitor or other display device. In some embodiments, the display device may be coupled directly to the system or processor generating or rendering the images. In other embodiments, the display device may be coupled indirectly to the system or processor such as via a network. Examples of such networks include the Internet, mobile telecommunications networks, a WIFI network, as well as any other wired and/or wireless networking system. When the display device is indirectly coupled, the images generated by the system or processor may be streamed over the network to the display device. Such streaming allows, for example, video games or other applications, which render images, to be executed on a server, a data center, or in a cloud-based computing environment and the rendered images to be transmitted and displayed on one or more user devices (such as a computer, video game console, smartphone, other mobile device, etc.) that are physically separate from the server or data center. Hence, the techniques disclosed herein can be applied to enhance the images that are streamed and to enhance services that stream images such as NVIDIA Geforce Now (GFN), Google Stadia, and the like.

Example Streaming System

[0140] FIG. **6** is an example system diagram for a streaming system **605**, in accordance with some embodiments of the present disclosure. FIG. **6** includes server(s) **603** (which may include similar components, features, and/or functionality to the example processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B), client device(s) **604** (which may include similar components, features, and/or functionality to the example processing system **500** of FIG. **5**A and/or exemplary system **565** of FIG. **5**B), and network(s) **606** (which may be similar to the network(s) described herein). In some embodiments of the present disclosure, the system **605** may be implemented.

[0141] In an embodiment, the streaming system **605** is a game streaming system and the server(s) **603** are game server(s). In the system **605**, for a game session, the client device(s) **604** may only receive input data in response to inputs to the input device(s) **626**, transmit the input data to the server(s) **603**, receive encoded display data from the server(s) **603**, and display the display data on the display **624**. As such, the more computationally intense computing and processing is offloaded to the server(s) **603** (e.g., rendering—in particular ray or path tracing—for graphical output of the game session is executed by the GPU(s) **615** of the server(s) **603**). In other words, the game session is streamed to the client device(s) **604** from the server(s) **603**, thereby reducing the requirements of the client device(s) **604** for graphics processing and rendering.

[0142] For example, with respect to an instantiation of a game session, a client device **604** may be displaying a frame of the game session on the display **624** based on receiving the display data from the server(s) **603**. The client device **604** may receive an input to one of the input device(s) **626** and generate input data in response. The client device **604** may transmit the input data to the server(s) **603** via the communication interface **621** and over the network(s) **606** (e.g., the Internet), and the server(s) **603** may receive the input data via the communication interface **618**. The CPU(s) **608** may receive the input data, process the input data, and transmit data to the GPU(s) **615** that causes the GPU(s) **615** to generate a rendering of the game session. For example, the input data may be representative of a movement of a character of the user in a game, firing a weapon, reloading, passing a ball, turning a vehicle, etc. The rendering component **612** may render the game session (e.g., representative of the result of the input data) and the render capture component **614** may capture the rendering of the game session as display data (e.g., as image data capturing the rendered frame of the game session). The rendering of the game session may include ray or path-traced lighting and/or shadow effects, computed using one or more parallel processing units—such as GPUs, which may further employ the use of one or more dedicated hardware accelerators or processing cores to perform ray or path-tracing techniques—of the server(s) **603**. The encoder **616** may then encode the display data to generate encoded display data and the encoded display data may be transmitted to the client device **604** over the network(s) **606** via the communication interface **618**. The client device **604** may receive the encoded display data via the communication interface **621** and the decoder **622** may decode the encoded display data to generate the display data. The client device **604** may then display the display data via the display **624**.

[0143] It is noted that the techniques described herein may be embodied in executable instructions stored in a computer readable medium for use by or in connection with a processor-based instruction execution machine, system, apparatus, or device. It will be appreciated by those skilled in the art that, for some embodiments, various types of computer-readable media can be included for storing data. As used

herein, a "computer-readable medium" includes one or more of any suitable media for storing the executable instructions of a computer program such that the instruction execution machine, system, apparatus, or device may read (or fetch) the instructions from the computer-readable medium and execute the instructions for carrying out the described embodiments. Suitable storage formats include one or more of an electronic, magnetic, optical, and electromagnetic format. A non-exhaustive list of conventional exemplary computer-readable medium includes: a portable computer diskette; a random-access memory (RAM); a read-only memory (ROM); an erasable programmable read only memory (EPROM); a flash memory device; and optical storage devices, including a portable compact disc (CD), a portable digital video disc (DVD), and the like.

[0144] It should be understood that the arrangement of components illustrated in the attached Figures are for illustrative purposes and that other arrangements are possible. For example, one or more of the elements described herein may be realized, in whole or in part, as an electronic hardware component. Other elements may be implemented in software, hardware, or a combination of software and hardware. Moreover, some or all of these other elements may be combined, some may be omitted altogether, and additional components may be added while still achieving the functionality described herein. Thus, the subject matter described herein may be embodied in many different variations, and all such variations are contemplated to be within the scope of the claims.

[0145] To facilitate an understanding of the subject matter described herein, many aspects are described in terms of sequences of actions. It will be recognized by those skilled in the art that the various actions may be performed by specialized circuits or circuitry, by program instructions being executed by one or more processors, or by a combination of both. The description herein of any sequence of actions is not intended to imply that the specific order described for performing that sequence must be followed. All methods described herein may be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context.

[0146] The use of the terms "a" and "an" and "the" and similar references in the context of describing the subject matter (particularly in the context of the following claims) are to be construed to cover both the singular and the plural, unless otherwise indicated herein or clearly contradicted by context. The use of the term "at least one" followed by a list of one or more items (for example, "at least one of A and B") is to be construed to mean one item selected from the listed items (A or B) or any combination of two or more of the listed items (A and B), unless otherwise indicated herein or clearly contradicted by context. Furthermore, the foregoing description is for the purpose of illustration only, and not for the purpose of limitation, as the scope of protection sought is defined by the claims as set forth hereinafter together with any equivalents thereof. The use of any and all examples, or exemplary language (e.g., "such as") provided herein, is intended merely to better illustrate the subject matter and does not pose a limitation on the scope of the subject matter unless otherwise claimed. The use of the term "based on" and other like phrases indicating a condition for bringing about a result, both in the claims and in the written description, is not intended to foreclose any other conditions that bring about that result. No language in the specification should be construed as indicating any non-claimed element as essential to the practice of the invention as claimed.

What is claimed is:

1. A processor comprising:

one or more arithmetic logic units (ALUs) configured to perform, using one or more neural networks, seed trajectory generation for optimization-based motion planning, the one or more neural networks comprising:

an observation encoder configured to:

receive a representation of an environment, and

generate, by encoding the representation of the environment, an environment embedding; and

a noise prediction network, configured to:

receive the environment embedding, a representation of an initial state, and a representation of a target state, and

generate a plurality of first trajectories based on the representation of the initial state, the representation of the target state, and random noise, and

generate a plurality of seed trajectories by denoising, in parallel, the plurality of first trajectories based on the environment embedding.

2. The processor according to claim **1**, wherein the observation encoder comprises a vision transformer, and wherein the representation of the environment comprises depth image data and camera properties corresponding to the depth image data.

3. The processor according to claim **2**, wherein the observation encoder comprises:

a first network configured to encode the depth image data,

a second network configured to encode the camera properties, and

wherein the observation encoder is configured to generate the environment embedding based on the encoded depth image data and the encoded camera properties.

4. The processor according to claim **2**, wherein the observation encoder comprises a multi-layer perceptron (MLP) configured to:

receive, as input, the camera properties corresponding to the depth image data, and

encode the camera properties to generate a vector representation of the camera properties, and

wherein the observation encoder is configured to generate the environment embedding by encoding the depth image data and the vector representation of the camera properties to generate an environment vector.

5. The processor according to claim **1**, wherein the environment embedding comprises an environment vector,

wherein the noise prediction network comprises a multi-layer perceptron (MLP), the MLP being configured to:

receive the representation of the initial state and the representation of the target state, and

encode the representation of the initial state and the representation of the target state to generate a trajectory endpoints vector, and

wherein the noise prediction network is configured to concatenate the environment vector and the trajectory endpoints vector to generate a concatenated environment vector.

6. The processor according to claim **1**, wherein the noise prediction network is configured to generate the plurality of first trajectories based on the representation of the initial state, the representation of the target state, and the random noise by:

encoding the representation of the initial state and the representation of the target state with time steps to generate a problem embedding,

generating a plurality of noise tensors of predefined dimensions, each noise tensor of the plurality of noise tensors corresponding to a first trajectory of the plurality of first trajectories, and

generating each first trajectory of the plurality of first trajectories based on the problem embedding and a respective noise tensor of the plurality of noise tensors.

7. The processor according to claim **1**, wherein the noise prediction network comprises a diffusion model, and

wherein the denoising, in parallel, the plurality of first trajectories based on the environment embedding comprises:

instantiating a plurality of instances of the diffusion model, each instance of the diffusion model corresponding to a first trajectory of the plurality of first trajectories,

providing, to each instance of the plurality of instances of the diffusion model, the environment embedding and a corresponding first trajectory of the plurality of first trajectories, and

denoising, by each instance of the diffusion model based on the environment embedding, the corresponding first trajectory.

8. The processor according to claim **7**, wherein the denoising, by each instance of the diffusion model based on the environment embedding, the corresponding first trajectory is performed iteratively, wherein an input to a subsequent iteration of the denoising operation is generated based on an output of a previous iteration of the denoising operation.

9. The processor according to claim **7**, wherein the diffusion model is a conditional diffusion model having a UNet architecture.

10. The processor according to claim **1**, wherein the one or more ALUs are further configured to perform, by an optimization-based motion planner, motion planning,

wherein the optimization-based motion planner is configured to:

receive, as input, the plurality of seed trajectories, and

generate, by performing optimization on the plurality of seed trajectories, a final motion plan from the initial state to the target state.

11. The processor according to claim **10**, wherein the motion planning is robot motion planning for a robot, wherein the initial state is a joint configuration of the robot, wherein the target state is defined by a pose of a tool of the robot, and wherein each seed trajectory of the plurality of seed trajectories provides a path from the initial state to the target state.

12. The processor according to claim **1**, wherein the observation encoder and the noise prediction network are jointly trained via a joint training procedure, the joint training procedure comprising:

generating, based on a ground-truth trajectory and a random noise, a first training trajectory,

generating, by denoising the first trajectory based on a training environment embedding, a second training trajectory,

determining, based on the second training trajectory and the ground-truth trajectory, a loss, and

jointly updating, based on gradients of the loss, learnable weights of the observation encoder and learnable weights of the noise prediction network.

13. A system, comprising:

one or more processors configured to perform, using one or more neural networks, seed trajectory generation for optimization-based motion planning, the one or more neural networks comprising:

an observation encoder configured to:

receive a representation of an environment, and

generate, by encoding the representation of the environment, an environment embedding; and

a noise prediction network, configured to:

receive the environment embedding, a representation of an initial state, and a representation of a target state,

generate a plurality of first trajectories based on the representation of the initial state, the representation of the target state, and random noise, and

generate a plurality of seed trajectories by denoising, in parallel, the plurality of first trajectories based on the environment embedding; and

one or more memories to store parameters associated with the one or more neural networks.

14. The system according to claim **13**, wherein the observation encoder comprises a vision transformer, and wherein the representation of the environment comprises depth image data and camera properties corresponding to the depth image data.

15. The system according to claim **14**, wherein the observation encoder comprises:

a first network configured to encode the depth image data,

a second network configured to encode the camera properties, and

wherein the observation encoder is configured to generate the environment embedding based on the encoded depth image data and the encoded camera properties.

16. The system according to claim **14**, wherein the observation encoder comprises a multi-layer perceptron (MLP) configured to:

receive, as input, the camera properties corresponding to the depth image data, and

encode the camera properties to generate a vector representation of the camera properties, and

wherein the observation encoder is configured to generate the environment embedding by encoding the depth image data and the vector representation of the camera properties to generate an environment vector.

17. The system according to claim **13**, wherein the environment embedding comprises an environment vector,

wherein the noise prediction network comprises a multi-layer perceptron (MLP), the MLP being configured to:

receive the representation of the initial state and the representation of the target state, and

encode the representation of the initial state and the representation of the target state to generate a trajectory endpoints vector, and

wherein the noise prediction network is configured to concatenate the environment vector and the trajectory endpoints vector to generate a concatenated environment vector.

18. The system according to claim **13**, wherein the noise prediction network is configured to generate the plurality of

first trajectories based on the representation of the initial state, the representation of the target state, and the random noise by:

encoding the representation of the initial state and the representation of the target state with time steps to generate a problem embedding,

generating a plurality of noise tensors of predefined dimensions, each noise tensor of the plurality of noise tensors corresponding to a first trajectory of the plurality of first trajectories, and

generating each first trajectory of the plurality of first trajectories based on the problem embedding and a respective noise tensor of the plurality of noise tensors.

19. The system according to claim **13**, wherein the noise prediction network comprises a diffusion model, and

wherein the denoising, in parallel, the plurality of first trajectories based on the environment embedding comprises:

instantiating a plurality of instances of the diffusion model, each instance of the diffusion model corresponding to a first trajectory of the plurality of first trajectories,

providing, to each instance of the plurality of instances of the diffusion model, the environment embedding and a corresponding first trajectory of the plurality of first trajectories, and

denoising, by each instance of the diffusion model based on the environment embedding, the corresponding first trajectory.

20. The system according to claim **19**, wherein the denoising, by each instance of the diffusion model based on the environment embedding, the corresponding first trajectory is performed iteratively, wherein an input to a subsequent iteration of the denoising operation is generated based on an output of a previous iteration of the denoising operation.

21. The system according to claim **19**, wherein the diffusion model is a conditional diffusion model having a UNet architecture.

22. The system according to claim **13**, wherein the one or more processors are further configured to perform, by an optimization-based motion planner, the optimization-based motion planning,

wherein the optimization-based motion planner is configured to:

receive, as input, the plurality of seed trajectories, and

generate, by performing optimization on the plurality of seed trajectories, a final motion plan from the initial state to the target state.

23. The system according to claim **22**, wherein the motion planning is robot motion planning for a robot, wherein the initial state is a joint configuration of the robot, wherein the target state is defined by a pose of a tool of the robot, and wherein each seed trajectory of the plurality of seed trajectories provides a path from the initial state to the target state.

24. The system according to claim **13**, wherein the observation encoder and the noise prediction network are jointly trained via a joint training procedure, the joint training procedure comprising:

generating, based on a ground-truth trajectory and a random noise, a first training trajectory,

generating, by denoising the first trajectory based on a training environment embedding, a second training trajectory,

determining, based on the second training trajectory and the ground-truth trajectory, a loss, and

jointly updating, based on gradients of the loss, learnable weights of the observation encoder and learnable weights of the noise prediction network.

25. A machine-readable medium having stored thereon a set of instructions, which if performed by one or more processors, cause the one or more processors to:

perform, using one or more neural networks, seed trajectory generation for optimization-based motion planning, the one or more neural networks comprising:

an observation encoder configured to:

receive a representation of an environment, and

generate, by encoding the representation of the environment, an environment embedding; and

a noise prediction network, configured to:

receive the environment embedding, a representation of an initial state, and a representation of a target state, and

generate a plurality of first trajectories based on the representation of the initial state, the representation of the target state, and random noise, and

generate a plurality of seed trajectories by denoising, in parallel, the plurality of first trajectories based on the environment embedding.

26. The machine-readable medium according to claim **25**, wherein the observation encoder comprises a vision transformer, and wherein the representation of the environment comprises depth image data and camera properties corresponding to the depth image data.

27. The machine-readable medium according to claim **25**, wherein the noise prediction network is configured to generate the plurality of first trajectories based on the representation of the initial state, the representation of the target state, and the random noise by:

encoding the representation of the initial state and the representation of the target state with time steps to generate a problem embedding,

generating a plurality of noise tensors of predefined dimensions, each noise tensor of the plurality of noise tensors corresponding to a first trajectory of the plurality of first trajectories, and

generating each first trajectory of the plurality of first trajectories based on the problem embedding and a respective noise tensor of the plurality of noise tensors.

28. The machine-readable medium according to claim **25**, wherein the noise prediction network comprises a diffusion model, and

wherein the denoising, in parallel, the plurality of first trajectories based on the environment embedding comprises:

instantiating a plurality of instances of the diffusion model, each instance of the diffusion model corresponding to a first trajectory of the plurality of first trajectories,

providing, to each instance of the plurality of instances of the diffusion model, the environment embedding and a corresponding first trajectory of the plurality of first trajectories, and

denoising, by each instance of the diffusion model based on the environment embedding, the corresponding first trajectory.

29. The machine-readable medium according to claim **25**, wherein the set of instructions further include additional

instructions, which if performed by one or more processors, cause the one or more processors to:

perform, by an optimization-based motion planner, the optimization-based motion planning,

wherein the optimization-based motion planner is configured to:

receive, as input, the plurality of seed trajectories, and

generate, by performing optimization on the plurality of seed trajectories, a final motion plan from the initial state to the target state.

30. The machine-readable medium according to claim **25**, wherein the observation encoder and the noise prediction network are jointly trained via a joint training procedure, the joint training procedure comprising:

generating, based on a ground-truth trajectory and a random noise, a first training trajectory,

generating, by denoising the first trajectory based on a training environment embedding, a second training trajectory,

determining, based on the second training trajectory and the ground-truth trajectory, a loss, and

jointly updating, based on gradients of the loss, learnable weights of the observation encoder and learnable weights of the noise prediction network.

31. A robot, comprising:

one or more processors configured to perform, using one or more neural networks, seed trajectory generation for optimization-based motion planning, the one or more neural networks comprising:

an observation encoder configured to:

receive a representation of an environment, and

generate, by encoding the representation of the environment, an environment embedding; and

a noise prediction network, configured to:

receive the environment embedding, a representation of an initial state, and a representation of a target state, and

generate a plurality of first trajectories based on the representation of the initial state, the representation of the target state, and random noise, and

generate a plurality of seed trajectories by denoising, in parallel, the plurality of first trajectories based on the environment embedding.

32. The robot according to claim **31**, wherein the observation encoder comprises a vision transformer, and wherein the representation of the environment comprises depth image data and camera properties corresponding to the depth image data.

33. The robot according to claim **31**, wherein the noise prediction network is configured to generate the plurality of first trajectories based on the representation of the initial state, the representation of the target state, and the random noise by:

encoding the representation of the initial state and the representation of the target state with time steps to generate a problem embedding,

generating a plurality of noise tensors of predefined dimensions, each noise tensor of the plurality of noise tensors corresponding to a first trajectory of the plurality of first trajectories, and

generating each first trajectory of the plurality of first trajectories based on the problem embedding and a respective noise tensor of the plurality of noise tensors.

34. The robot according to claim **31**, wherein the noise prediction network comprises a diffusion model, and

wherein the denoising, in parallel, the plurality of first trajectories based on the environment embedding comprises:

instantiating a plurality of instances of the diffusion model, each instance of the diffusion model corresponding to a first trajectory of the plurality of first trajectories,

providing, to each instance of the plurality of instances of the diffusion model, the environment embedding and a corresponding first trajectory of the plurality of first trajectories, and

denoising, by each instance of the diffusion model based on the environment embedding, the corresponding first trajectory.

35. The robot according to claim **31**, wherein one or more processors are further configured to perform, by an optimization-based motion planner, the optimization-based motion planning,

wherein the optimization-based motion planner is configured to:

receive, as input, the plurality of seed trajectories, and

generate, by performing optimization on the plurality of seed trajectories, a final motion plan from the initial state to the target state.

36. The robot according to claim **31**, wherein the observation encoder and the noise prediction network are jointly trained via a joint training procedure, the joint training procedure comprising:

generating, based on a ground-truth trajectory and a random noise, a first training trajectory,

generating, by denoising the first trajectory based on a training environment embedding, a second training trajectory,

determining, based on the second training trajectory and the ground-truth trajectory, a loss, and

jointly updating, based on gradients of the loss, learnable weights of the observation encoder and learnable weights of the noise prediction network.

37. A method for generating seed trajectories for optimization-based motion planning, the method comprising:

receiving, via an observation encoder network, a representation of an environment;

generating, by encoding the representation of the environment via the observation encoder network, an environment embedding;

receiving, via a noise prediction network, the environment embedding, a representation of an initial state, and a representation of a target state;

generating, via the noise prediction network, a plurality of first trajectories by the noise prediction network and based on the representation of the initial state, the representation of the target state, and random noise; and

generating, via the noise prediction network, a plurality of seed trajectories by denoising, by the noise prediction network, the plurality of first trajectories, wherein the denoising the plurality of first trajectories is performed in parallel and based on the environment embedding.

38. The method according to claim **37**, wherein the observation encoder comprises a vision transformer, and wherein the representation of the environment comprises depth image data and camera properties corresponding to the depth image data.

39. The method according to claim **37**, wherein the generating the plurality of first trajectories comprises:

encoding the representation of the initial state and the representation of the target state with time steps to generate a problem embedding,

generating a plurality of noise tensors of predefined dimensions, each noise tensor of the plurality of noise tensors corresponding to a first trajectory of the plurality of first trajectories, and

generating each first trajectory of the plurality of first trajectories based on the problem embedding and a respective noise tensor of the plurality of noise tensors.

40. The method according to claim **37**, wherein the denoising the plurality of first trajectories comprises:

instantiating a plurality of instances of a diffusion model, each instance of the diffusion model corresponding to a first trajectory of the plurality of first trajectories,

providing, to each instance of the diffusion model, the environment embedding and a corresponding first trajectory of the plurality of first trajectories, and

denoising, by each instance of the diffusion model and based on the environment embedding, the corresponding first trajectory.

41. The method according to claim **37**, further comprising:

determining, by an optimization-based motion planner, a final motion plan, wherein determining the final motion plan comprises:

receiving, as input, the plurality of seed trajectories, and

generating, by performing optimization on the plurality of seed trajectories, the final motion plan.

42. The method according to claim **37**, wherein the observation encoder and the noise prediction network are jointly trained via a joint training procedure, the joint training procedure comprising:

generating, based on a ground-truth trajectory and a random noise, a first training trajectory,

generating, by denoising the first trajectory based on a training environment embedding, a second training trajectory,

determining, based on the second training trajectory and the ground-truth trajectory, a loss, and

jointly updating, based on gradients of the loss, learnable weights of the observation encoder and learnable weights of the noise prediction network.

* * * * *